SOFTWARE DESIGN DOCUMENT (SDD)		
PRESENTED BY:	**PRESENTED TO:**	**DATE:**
CodeCrafters	Universiti Malaysia Pahang Al-Sultan Abdullah	15 January 2025

TEAM MEMBERS

Chief Executive Officer	Chief Business Analyst	Chief Developer	Software Quality Assurance
			
MIRZA RUSYAIDI BIN RAHAMAT CB23124	AMALA KARTHIGAYAN CB23116	NUR ASYIKIN BINTI ISMAIL CB23121	NABILAH BINTI MOHAMMAD SHAHIDAN CB23122

Submitted to the Software Design Workshop (BCS2343)

Bachelor of Computer Science (Software Engineering) with Honors

Faculty of Computing, Universiti Malaysia Pahang Al-Sultan Abdullah (UMPSA)

DOCUMENT APPROVAL

	Name	Date
Authenticated by: _____ Project Leader	MIRZA RUSYAIDI BIN RAHAMAT Project Leader	15/1/2025
Approved by: _____ Project Manager	AMALA KARTHIGAYAN Project Manager	15/1/2025
Approved by: _____ Client		

Software : Adobe Acrobat DC, Microsoft Word

Archiving Place : https://shorturl.at/MlPfB

Copies Available : 1 Hardcopy

Table of Contents

LIST OF FIGURES

LIST OF ABBREVIATIONS

Software Design Document	SDD
Final Year Project	FYP
Primary Key	PK
Foreign Key	FK

21. INTRODUCTION

21.1. Purpose

The purpose of this document is to outline the specifications and requirements for the development of the Supervisor Hunting Application, designed to assist in the management of final year project (FYP) supervision for undergraduate students in the Faculty of Computing. The purpose of this document is to provide a comprehensive Software Design Document (SDD) for the Final Year Project (FYP) Supervisor Hunting Application, an application proposed by the Faculty of Computing, University Malaysia Pahang Al-Sultan Abdullah. The aim of this document is to provide an overview of all parties associated with the project that all expected readers, which are developers, owner of the system (Faculty of Computing), respective stakeholders such as students, lecturers, Final Year Project coordinators, head of research group, and team members have to make sure that everyone is aware of their obligations and functions, it sets forth the objectives and requirements of the system.

The document encompasses the Software Design Document (SDD). The SDD section ensures that the development team and stakeholders have a clear and common knowledge of what the system is designed to achieve by providing a full explanation of the functional and non-functional requirements of the system. This document's scope includes outlining project goals, like making it simpler for students can manage users, manage appointment, apply topic/title and manage time frame and quota. Then, other scopes are the system can manage notifications, giving real-time information on supervisor availability can view and monitor activities within the system.

21.2. System Identification

<table>
<tr><td>System Identification Number</td><td>1000 (SDW_SDD_FYPSHA 1.1.24)
The system identification number shows the identification number for Final Year Project (FYP) Supervisor Hunting Application</td></tr>
<tr><td>System Title</td><td>Final Year Project (FYP) Supervisor Hunting Application</td></tr>
<tr><td>System Abbreviation</td><td>FYPSHA</td></tr>
<tr><td>System Version Number</td><td>1.1.24
This is initial release number for Final Year Project (FYP) Supervisor Hunting Application</td></tr>
<tr><td>System Release Number</td><td>1.1.24
Start with number 1as the major release number and middle with number 1 as the minor release number that this is initial version of the system.
Number 24 as the service release that shows the system released in 2024.</td></tr>
<tr><td>Document Identification ID</td><td>SDD_FYPSHA_2024
Meaning of term:<table><tr><td>SDD</td><td>Software Design Document</td></tr><tr><td>FYPSHA</td><td>Final Year Project Supervisor Hunting Application</td></tr><tr><td>2024</td><td>Year of document created</td></tr></table></td></tr>
</table>

21.3. System Overview

The proposed Supervisor Hunting Application is designed to streamline the process of students in the Faculty of Computing finding supervisors for their Final Year Projects (FYP). This system aims to assist the FYP Coordinator in managing interactions between students and lecturers, ensuring an organized and efficient matching process based on project themes, student interests, and lecturer expertise.

The application serves various users: the Deputy Dean of Research, FYP Coordinator, FC Lecturers, and FC Students. Key features include setting schedules and announcements for supervisor hunting activities, managing supervisor quotas, allowing lecturers to post FYP topics, enabling students to search and communicate with potential supervisors, and registering agreed supervisors. The system also provides real-time updates on supervisor availability, suggests relevant supervisors for students who haven't identified one, and allows the HoRG and Coordinator to monitor the process and generate reports.

Overall, the Supervisor Hunting Application enhances efficiency, transparency, and communication in the supervisor selection process, ensuring every student finds a suitable supervisor for their FYP.

21.4. References

1. Brown, F. (2024, June 28). *Entity Relationship (ER) Diagram Model with DBMS Example*. Guru99. https://www.guru99.com/er-diagram-tutorial-dbms.html
2. GeeksforGeeks. (2024, July 8). *MVC Framework Introduction*. GeeksforGeeks. https://www.geeksforgeeks.org/mvc-framework-introduction/
3. GeeksforGeeks. (2024b, November 11). *Software Architectural Patterns in System Design*. GeeksforGeeks. https://www.geeksforgeeks.org/design-patterns-architecture/
4. DR. ROHANI BINTI ABU BAKAR," SOFTWARE REQUIREMENT SPECIFICATION (SRS)"
5. DR. IDAHAM ONG, "NOTICE FOR OPEN TENDER REQUEST FOR PROPOSAL, 2024"

21.5. Entity Relationship Diagram (ERD)

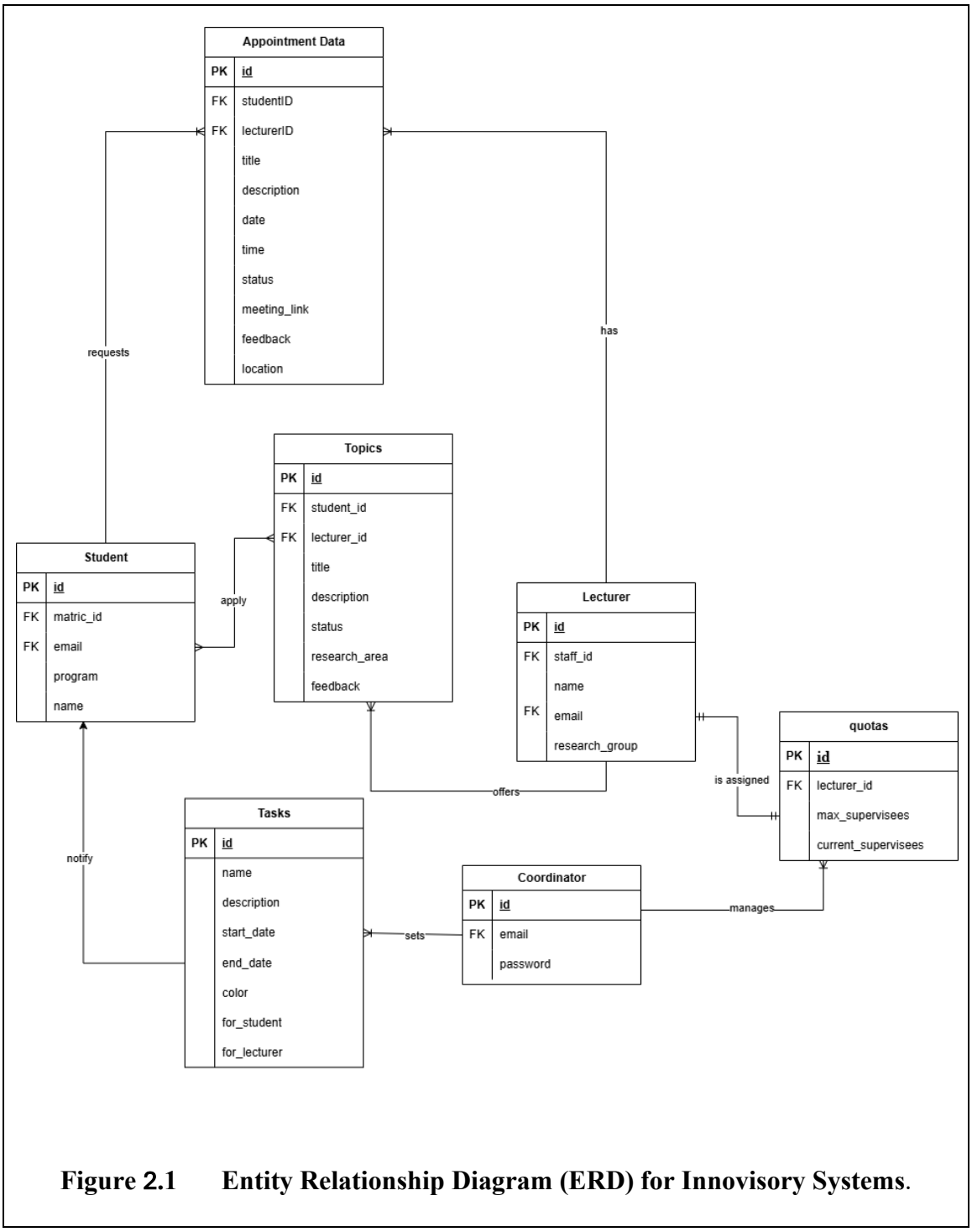


Figure 2.1 Entity Relationship Diagram (ERD) for Innovisory Systems.

21.6. Data Dictionary

2.2.1 Coordinator

Field Name	Description	Data Type	Constraint
id	coordinator id	BIGINT (20)	Primary Key (PK)
email	email of coordinator	VARCHAR (255)	Foreign Key (FK) REFERENCES Coordinator(id), NOT NULL
password	password	VARCHAR (255)	NOT NULL

2.2.2. Lecturer

Field Name	Description	Data Type	Constraint
id	Lecturer id	VARCHAR (255)	Primary Key (PK)
staff_id	staff id	VARCHAR (255)	Foreign Key (FK) REFERENCES Lecturer(id), NOT NULL
name	name of lecturer	VARCHAR (255)	NOT NULL
email	email of lecturer	VARCHAR (255)	Foreign Key (FK) REFERENCES Lecturer(id), NOT NULL
research_group	Number of assigned students	VARCHAR (255)	NO NULL

2.2.3. Student

Field Name	Description	Data Type	Constraint
id	Student id	BIGINT (20)	Primary Key (PK)
matric_id	matric id	VARCHAR (255)	Foreign Key (FK) REFERENCES Student(id), NOT NULL

name	Batch id	VARCHAR (255)	NOT NULL
email	Assigned lecturer	VARCHAR (255)	Foreign Key (FK) REFERENCES Student(id)
program	course	VARCHAR (255)	NOT NULL

2.2.4. Topics

Field Name	Description	Data Type	Constraint
id	Topic id	BIGINT (20)	Primary Key (PK)
student_id	Student id	BIGINT (20)	Foreign Key (FK) REFERENCES Student(student_id), NOT NULL
lecturer_id	Lecturer id	BIGINT (20)	Foreign Key (FK) REFERENCES Lecturer(lecturer_id), NULL
title	Project title for project topic	VARCHAR (255)	NOT NULL
description	Description of project title	TEXT	NULL
research_area	scope of research	VARCHAR (255)	NOT NULL
status	Status of project topic (‘pending', 'approved', 'rejected')	ENUM	NOT NULL
feedback	feedback of topics	TEXT	NULL

2.2.5. Quota

Field Name	Description	Data Type	Constraint
id	quota id	BIGINT (20)	Primary Key (PK)

lecturer_id	Lecturer id	BIGINT (20)	Foreign Key (FK) REFERENCES Quota(id)
max_supervisees	maximum of supervisees	INT (11)	NOT NULL
current_supervisees	current supervisees	INT (11)	NOT NULL

2.2.6. Tasks

Field Name	Description	Data Type	Constraint
id	tasks id	BIGINT (20)	Primary Key (PK)
name	name of task	VARCHAR (255)	NOT NULL
description	description of task	TEXT	NULL
start_date	Start date of timeframe	DATETIME	NOT NULL
end_date	End date of timeframe	DATETIME	NOT NULL
color	color of task	VARCHAR (255)	NOT NULL
for_student	timeframe user (student)	BINARY (1)	NOT NULL
for_lecturer	timeframe user (student)	BINARY (1)	NOT NULL

2.2.7. Appointment

Field Name	Description	Data Type	Constraint
id	Appointment id	BIGINT (20)	Primary Key (PK)
studentID	Student id	BIGINT (20)	Foreign Key (FK) REFERENCES Appointment(id)
lecturerID	Lecturer id	BIGINT (20)	Foreign Key (FK) REFERENCES Appointment (id)
title	Title of appointment	VARCHAR (255)	NO NULL
description	Description of appointment	TEXT	NO NULL
date	Date of appointment	DATE	NOT NULL

time	Time of appointment	TIME	NOT NULL
status	Status in (pending, approved, rejected, cancelled)	ENUM (pending', 'approved', 'rejected', 'completed)	pending
meeting_link	Remarks of appointment	VARCHAR (255)	NULL
feedback	feedback of appointment	TEXT	NULL
location	location of meeting	VARCHAR (255)	NULL

3. GENERAL ARCHITECTURE

3.1 Architectural Pattern: MVC

The system utilizes the design pattern known as Model-View-Controller (MVC). The architecture of this system is composed of three distinct layers, which are as follows:

1) **Model**: Represents the application's business logic and data handling.
2) **View**: Manages the user interface and displays data from the model.
3) **Controller**: Acts as a bridge between the View and Model, handling user input and application logic.

The MVC pattern's modular structure allows the system to be highly maintainable and scalable. Changes to one component, such as altering the View's design or updating the Model's database schema, can be made without significantly impacting the others. Additionally, the pattern is particularly well-suited for object-oriented programming, as each component can leverage encapsulation, inheritance. and polymorphism to streamline functionality and ensure code reuse. This makes MVC an ideal choice for the "FYP Supervisor Hunting" system, where multiple user roles and modules must function cohesively and efficiently.

3.2 Component Overview

The "FYP Supervisor Hunting" system uses a modular architecture, dividing the overall functionality into distinct, independent modules. Each module focuses on a specific aspect of the system, allowing for autonomous function and seamless interaction with other modules. The created independent components are Manage Users Module, Manage Appointments Module, Apply Topic Module, and Manage Timeframe and Quota Module.

31.1. Module 1: Manage Users

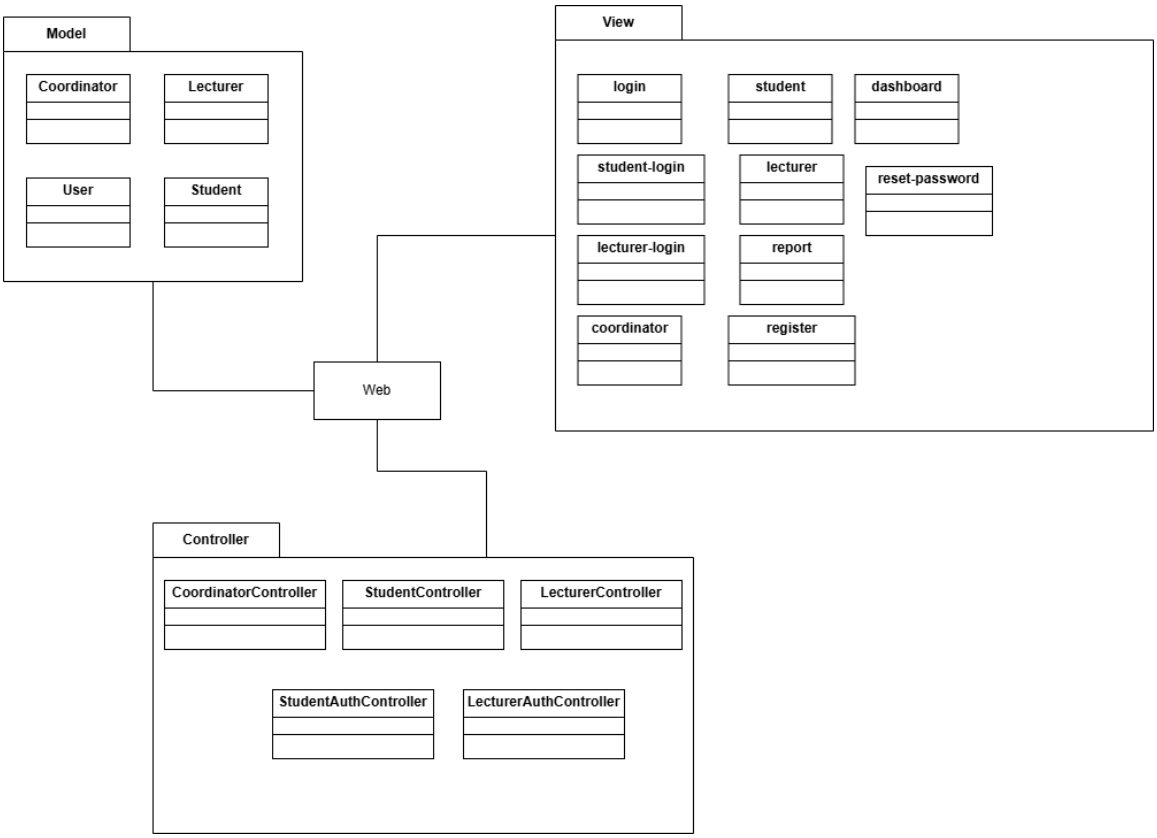


Figure 3.2.1 Component Module 1: Manage Users

3.2.1 Module 2: Manage Appointment

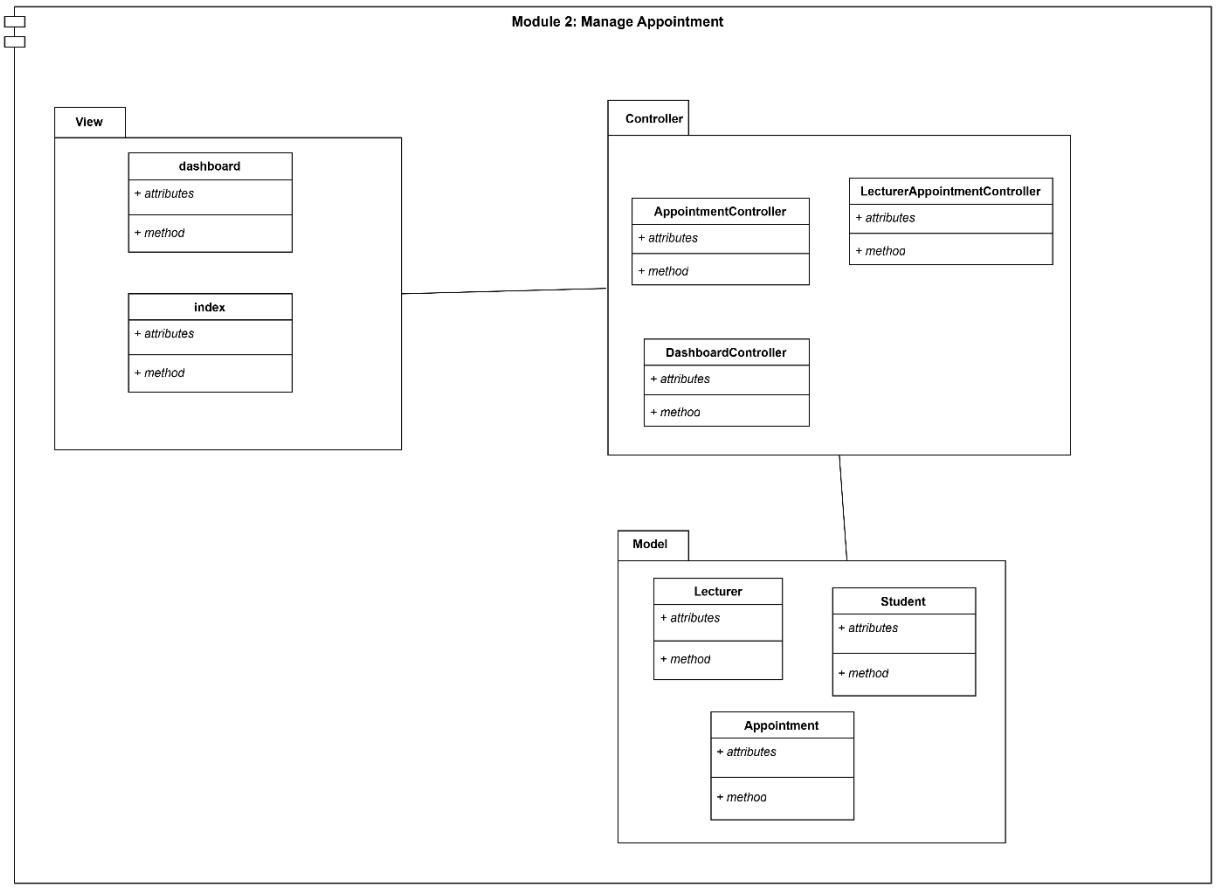


Figure 3.2.2 Component Module 2: Manage Appointment

View Package

Class Name	Description
dashboard	A class that controls the user to view the Lecturer and Student Dashboard.
index	A class that controls the user to view Manage Appointment for Lecturer and Student.

Controller Package

Class Name	Description
Appointment Controller	The intermediary class controls managing appointments.
LecturerAppointmentController	The intermediary class controls for lecturer appointments.

StudentController	The intermediary class controls for managing student-related operations.
DashboardController	The intermediary class controls for managing dashboard operations.
LecturerController	The intermediary class controls for lecturer-specific actions.

Model Package

Class Name	**Description**
Lecturer	The model class to handle operations related to lecturers, such as retrieving or updating lecturer information.
Student	The model class for operations related to students such as apply appointments and fetching student details.
Appointment	The model class for managing appointments, such as scheduling, updating, or canceling.

3.2.2 Module 3: Apply Topic/Title

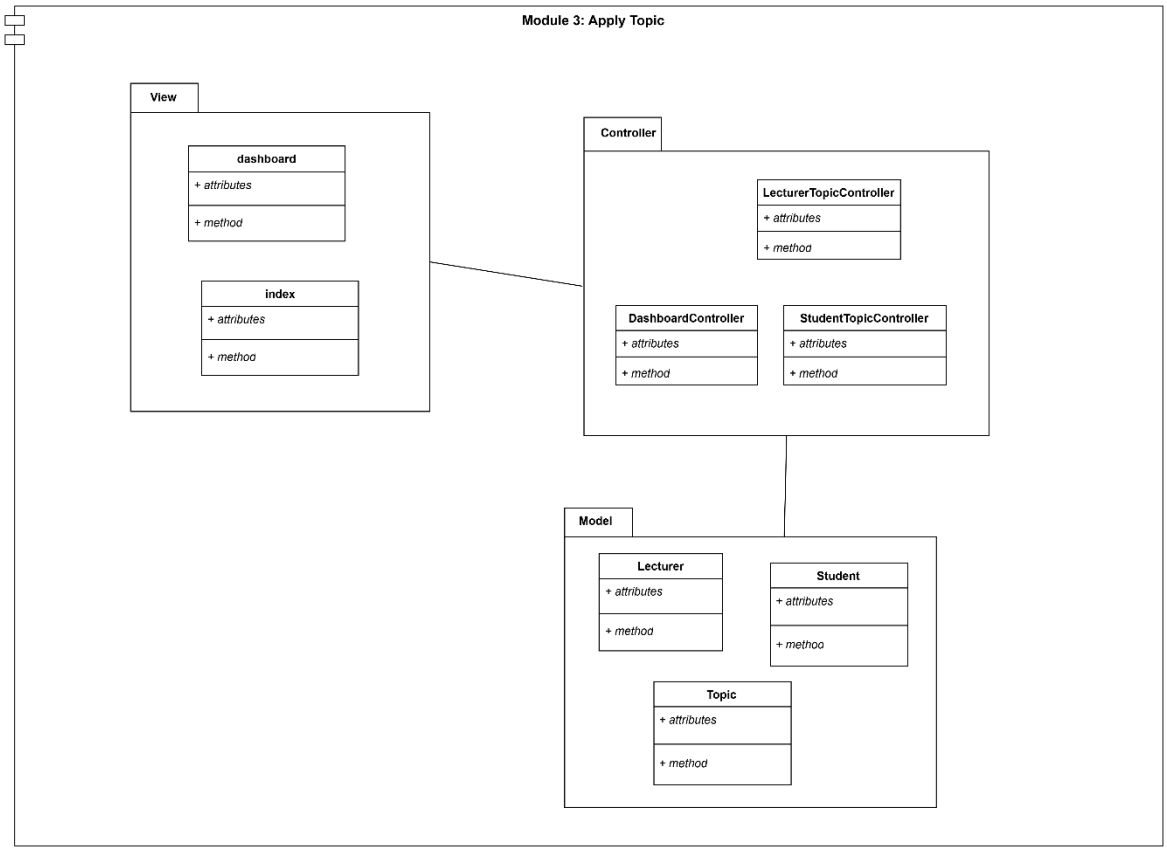


Figure 3.2.3 Component Module 3: Apply topic/Title

View Package

Class Name	Description
FYPTopicListUI	A class for displaying the FYP topic list and form for student to apply the topic.
FYPApplicationUI	A class for displaying the application made by student and lecturer can respond to their application.

Controller Package

Class Name	Description
FYPTopicController	A controller class for managing all the FYP topic proposed.

Model Package

Class Name	Description
TopicModel	A model class for handling and coordinating data related to the FYP topic.

51.1. Module 4: Manage Timeframe and Quota

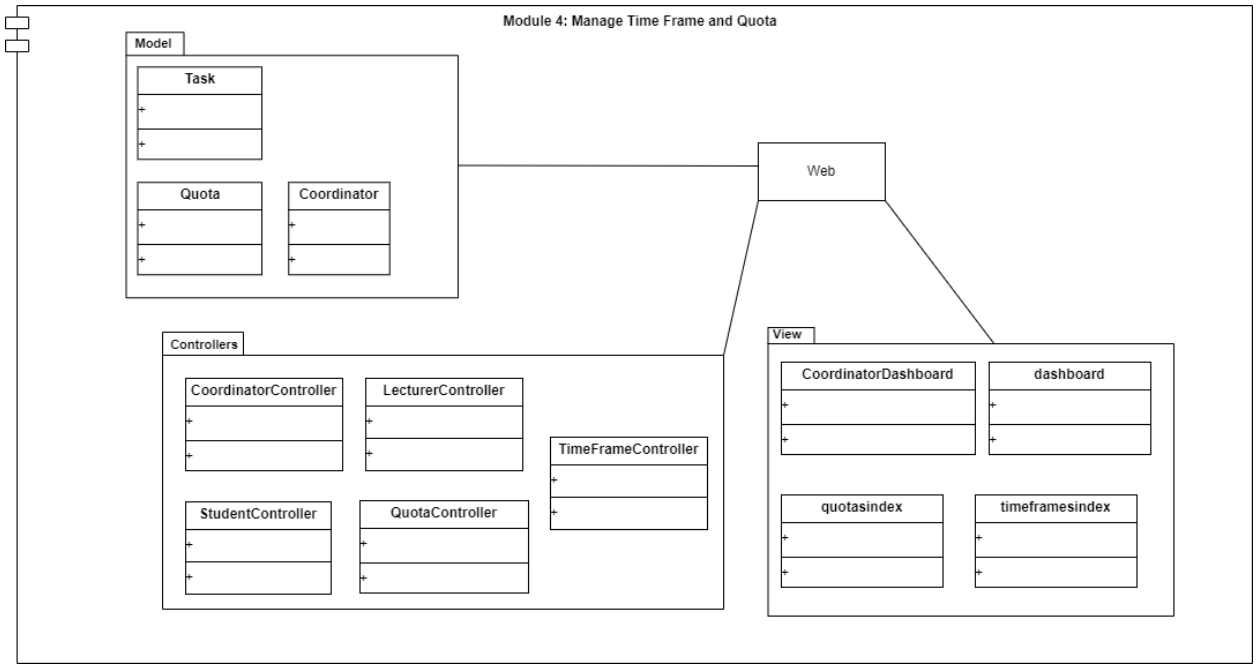


Figure 3.2.4 Component Module 4: Manage Timeframe and Quota

View Package

Class Name	Description
CoordinatorDashboard	A class for displaying the coordinator's dashboard interface, including an overview of time frames, quotas, and lecturer assignments.
Timeframesindex	A class for managing and displaying the list of all defined time frames, including options to add, update, or delete entries.
Quotasindex	A class for presenting a view to manage lecturer quotas, allowing the coordinator to set or adjust quotas interactively.
Dashboard	A general-purpose class for displaying key system data to lecturers and students, including notifications of updates to quotas or time frames.

Controller Package

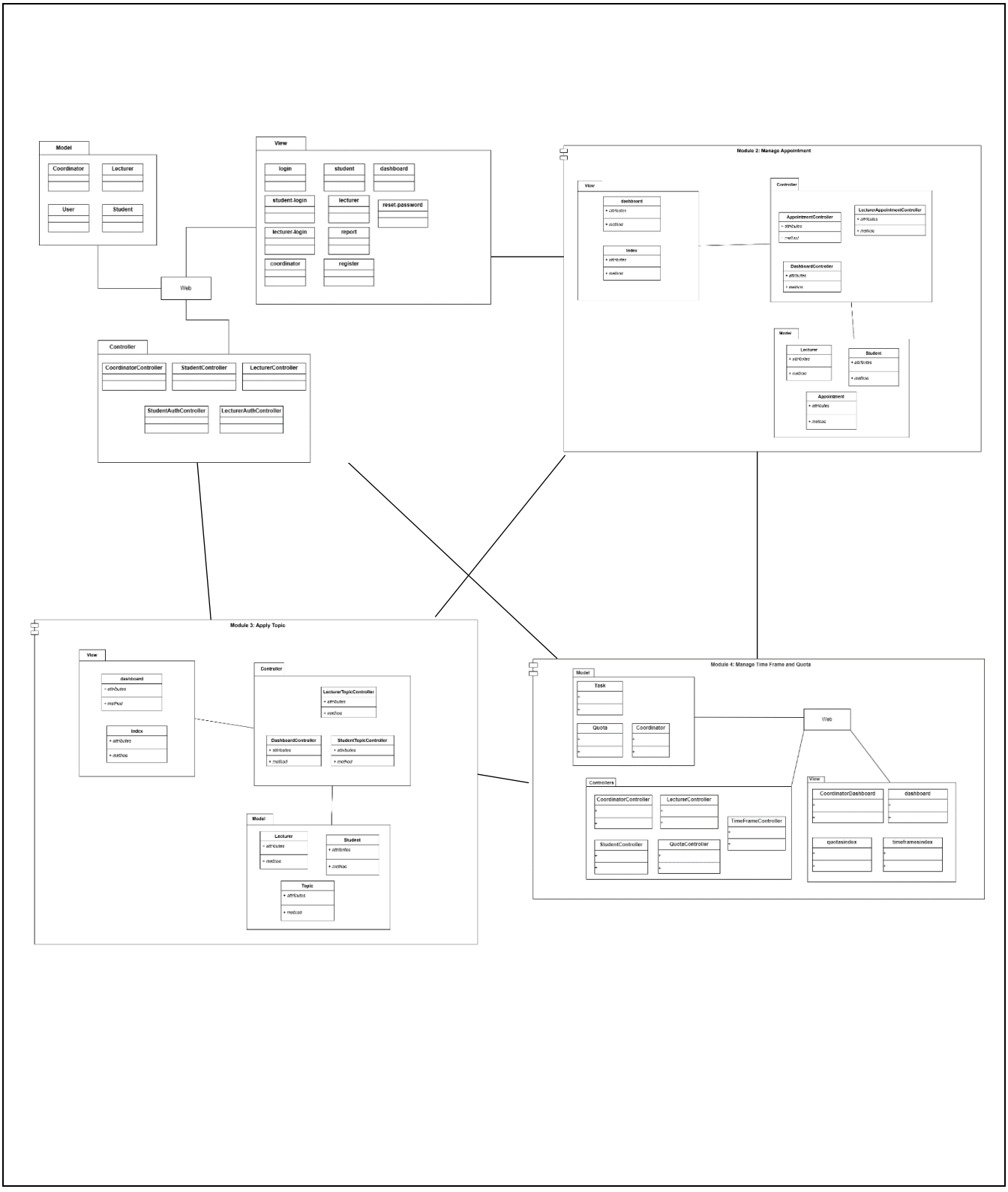
Class Name	Description
TimeFrameController	A controller class dedicated to managing operations related to time frames, including adding, updating, and validating timeframe data.
QuotaController	A controller class for handling operations related to quotas, including setting and updating lecturer supervision limits.
CoordinatorController	A controller class responsible for managing actions related to the coordinator, including modifying time frames and quotas.

Model Package

Class Name	Description
Coordinator	A model class representing the coordinator's actions, such as overseeing quotas and timeframe updates, and managing system settings.
Task	A model class for handling generic tasks related to managing time frames, and validating time frame details, Represents a timeframe entity. Manages data such as start_date, end_date.
Quota	A model class for storing and managing data related to lecturer quotas.

3.3 PACKAGE RELATIONSHIP

Figure 3.3.1 Package relationship



4. DETAIL DESIGN

4.1 Package 1 [SDD-REQ-100] (Module: Manage User)

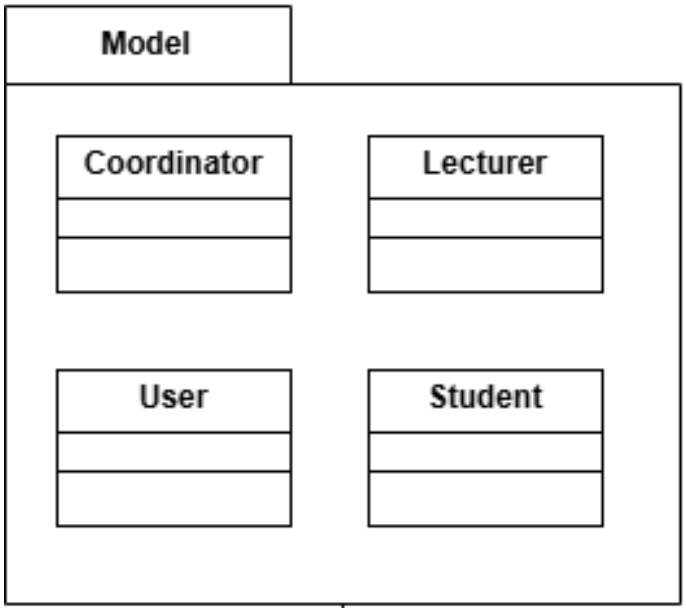


Figure 4.1 Model Manage User

4.1.1 [SDD-REQ-101] Coordinator.php

Class Type	Model	
Responsibility	• Represent and manage the data for a coordinator in the system. • Handle coordinator authentication, including password storage and notifications. • Define attributes and their behavior (e.g., hidden).	
Attributes	**Attributes Name**	**Attributes Type**
	email	string
	password	String(hidden)

	remember_token	string(nullable)
Methods	**Method Name**	**Description**
	notify()	Provides notification capabilities for the coordinator, allowing them to receive alerts or messages.
Algorithm	1. **Start** 2. Define the Coordinator class, extending Authenticatable to manage coordinator authentication. 3. Specify the fillable attributes: • email • password 4. Mark the password and remember_token attributes as hidden to ensure they are not included in array or JSON responses. 5. Enable notification functionality using the Notifiable trait to allow the coordinator to receive notifications. 6. **End**	

4.1.2 [SDD-REQ-102] Lecturer.php

Class Type	Model	
Responsibility	• Represent and manage the data for a lecturer in the system. • Handle lecturer authentication, including password storage and first login status. • Define attributes and their behavior (e.g., casts, hidden). • Manage the relationship with the Quota model.	
Attributes	**Attributes Name**	**Attributes Type**
	staff_id	string
	name	string
	email	string
	research_group	string
	password	string
	is_first_login	boolean
Methods	**Method Name**	**Description**
	quota()	Defines a one-to-one relationship between Lecturer and Quota. This allows access to the

		lecturer's assigned quota data.
Algorithm	1. **Start** 2. Define the Lecturer class, extending the Authenticatable class. 3. Specify the fillable attributes: staff_id name email research_group password is_first_login 4. Mark the password attribute as hidden to protect it in JSON responses. 5. Cast the is_first_login attribute to a boolean for easier management. 6. Define a one-to-one relationship with the Quota model using the quota() method. 7. **End**	

4.1.3 [SDD-REQ-103] User.php

Class Type	Model
Responsibility	• Represent and manage the data for a user in the system. • Handle user authentication, including password storage and two-factor authentication. • Manage user-related data such as name and email.

Attributes	**Attributes Name**	**Attributes Type**
	name	String
	email	String
	password	String (Hidden)
	remember_token	String (Hidden)
	two_factor_recovery_codes	String (Hidden)
	two_factor_secret	String (Hidden)
	profile_photo_url	String (Appended)
	email_verified_at	DateTime
Methods	**Method Name**	**Description**
	casts()	Defines the casting for specific attributes (email_verified_at as DateTime, password as hashed) to format the data appropriately.
Algorithm	1. **Start** 2. Define the User class, extending Authenticatable and including various traits for additional functionality (e.g., HasApiTokens, HasFactory, HasProfilePhoto, Notifiable, TwoFactorAuthenticatable).	

	3. Specify the fillable attributes: • name • email • password • Mark the following attributes as hidden: • password • remember_token • two_factor_recovery_codes • two_factor_secret 4. Define the casts() method to convert attributes as needed, such as: • email_verified_at to datetime • password to hashed 5. **End**

4.1.4 [SDD-REQ-104] Student.php

Class Type	Model	
Responsibility	• Represent and manage the data for a student in the system. • Handle student authentication, including password storage and first login status. • Define attributes and their behavior (e.g., casts, hidden).	
Attributes	**Attributes Name**	**Attributes Type**
	matric_id	String

	name	String
	email	String
	program	String
	password	String (Hidden)
	is_first_login	Boolean
Methods	**Method Name**	**Description**
	quota()	Defines a one-to-one relationship between Student and Quota. This allows access to the student's assigned quota data.
Algorithm	1. **Start** 2. Define the Student class, extending Authenticatable to manage student authentication. 3. Specify the fillable attributes: 1. matric_id 2. name 3. email 4. program	

	5. password 6. is_first_login 4. Mark the password attribute as hidden to ensure it's not included in array or JSON responses. 5. Cast the is_first_login attribute to a boolean for proper data handling. 6. Define the quota() method to establish a one-to-one relationship with the Quota model. 7. **End**

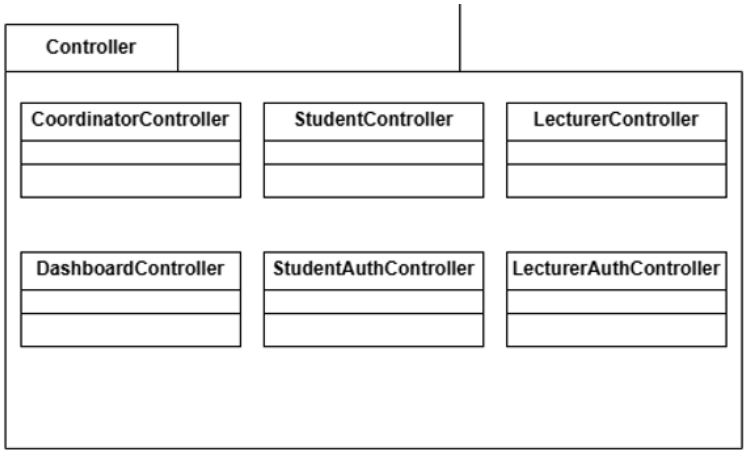


Figure 4.2 Controller Manage user

4.1.5 [SDD-REQ-105] CoordinatorController.php

Class Type	Controller	
Responsibility	• Handle the logic for the coordinator's authentication and dashboard functionalities. • Provide methods for showing the login form, handling login and logout, and showing the coordinator dashboard with relevant data.	
Attributes	**Attributes Name**	**Attributes Type**
	$totalStudents	Integer
	$totalLecturers	Integer
	$studentDistribution	Array
	$researchGroups	Array
Methods	**Method Name**	**Description**
	showLoginForm()	Displays the login form for the coordinator.

	login(Request $request)	Validates the coordinator's credentials and logs them in, redirecting to the dashboard if successful.
	logout(Request $request)	Logs the coordinator out, invalidates the session, and regenerates the CSRF token.
	showDashboard()	Retrieves and displays important data for the coordinator, such as the total number of students and lecturers, and program distribution. Also shows research group distribution and associated members.
Algorithm	1. **Start** 2. Define the CoordinatorController class, which handles the coordinator's authentication and dashboard logic. 3. Implement the showLoginForm() method to return the login view for the coordinator. 4. Implement the login() method: a. Validate the email and password from the request.	

	b. Use Auth::guard('coordinator') to attempt login with the given credentials. c. If successful, regenerate the session and redirect the coordinator to their dashboard. d. If authentication fails, return an error message. 5. Implement the logout() method: a. Log out the coordinator, invalidate the session, and regenerate the CSRF token. b. Redirect the user to the home page. 6. Implement the showDashboard() method: a. Retrieve data such as the total number of students, lecturers, student distribution by program, and the number of lecturers in each research group. b. Pass this data to the dashboard view to display it. 7. **End**

4.1.6 [SDD-REQ-106] StudentController.php

Class Type	Controller

Responsibility	• Manage student data, including listing, adding, updating, and deleting student records. • Import student data from a CSV file with validation for matric ID and program mapping. • Generate and display reports for all students. • Provide a downloadable template for student data. • Handle AJAX requests for editing and updating student records.	
Attributes	**Attributes Name**	**Attributes Type**
	programMapping	array
	students	object
Methods	**Method Name**	**Description**
	index()	Displays a paginated list of students.
	importCSV()	Imports student data from a CSV file.
	generateReport()	Generates and displays a student report.
	downloadTemplate()	Provides a downloadable CSV template for student data.
	destroy()	Deletes a student record.

	edit()	Fetches data for editing a student record.
	update()	Updates a student record with new data.
Algorithm	1. Start 2. Define the StudentController class with necessary methods for importing, exporting, and managing students. 3. Map matriculation ID prefixes to specific programs. 4. Create validation for CSV files with proper format and content. 5. Process the CSV file, check for errors, duplicates, and correct matriculation IDs. 6. Handle exceptions and errors during the import process. 7. Update student records and return success or failure messages as needed. 8. End	

4.1.7 [SDD-REQ-107] LecturerController.php

Class Type	Controller
Responsibility	• Import lecturer data from a CSV file, ensuring validation and error handling. • Allow downloading of a CSV template for lecturers. • Generate reports for all lecturers.

Attributes	**Attributes Name**	**Attributes Type**
	staff_id	string
	name	string
	email	string
	research_group	string
Methods	**Method Name**	**Description**
	index()	Retrieve and display a paginated list of lecturers.
	importCSV()	Import lecturer data from a CSV file.
	generateReport()	Download a CSV template for lecturers.
	downloadTemplate()	Generate a report for all lecturers.
	destroy()	Delete a lecturer from the system.
	edit()	Retrieve lecturer data for editing.
	update()	Update lecturer details.

Algorithm	1. Start 2. Define the LecturerController class extending the Controller base class. 3. Declare a private variable for mapping research groups. 4. Create methods for handling CRUD operations and CSV import/export. 5. Implement logic to validate, parse, and store CSV data during import. 6. Define methods to generate reports and templates. 7. End

4.1.8 [SDD-REQ-108] LecturerAuthController.php

Class Type	Controller	
Responsibility	- Manage authentication for lecturers. - Handle login, password change, and logout functionality for lecturers. - Redirect lecturers to appropriate views based on login status.	
Attributes	**Attributes Name**	**Attributes Type**
	lecturer	object
	request	object
Methods	**Method Name**	**Description**

	showLoginForm	Display the login form for lecturers.
	login	Authenticate the lecturer and redirect appropriately.
	showChangePasswordForm	Display the form to change the lecturer's password.
	changePassword	Handle password change for the lecturer.
	logout	Log out the lecturer and redirect to the login page.
Algorithm	1. Start 2. Define the `LecturerAuthController` class, extending the `Controller` base class. 3. Implement the `showLoginForm` method to return the lecturer login view. 4. Implement the `login` method: - Validate credentials (`staff_id` and `password`). - Attempt authentication using `Auth::guard('lecturer')`. - Redirect to the password change page if it is the lecturer's first login. - Otherwise, redirect to the lecturer dashboard. - Return back with errors if authentication fails. 5. Implement the `showChangePasswordForm` method to return the password change view. 6. Implement the `changePassword` method:	

	- Validate the input for current password and new password. - Verify the current password matches the stored password. - Update the password and set `is_first_login` to `false`. - Save the lecturer's updated record. - Redirect to the dashboard with a success message. 7. Implement the `logout` method to log out the lecturer and redirect to the login page. 8. End

4.1.9 [SDD-REQ-109] StudentAuthController.php

Class Type	Controller	
Responsibility	- Manage authentication for students. - Handle login, password change, and logout functionality for students. - Display dashboard data including tasks, pending topics, and appointments.	
Attributes	**Attributes Name**	**Attributes Type**
	student	object
	tasks	array

	pendingTopicCount	integer
	pendingAppointmentCount	integer
	request	object
Methods	**Method Name**	**Description**
	showLoginForm	Display the login form for students.
	login	Authenticate the student and redirect appropriately.
	showChangePasswordForm	Display the form to change the student's password.
	changePassword	Handle password change for the student.
	logout	Log out the student and redirect to the login page.
	dashboard	Fetch and display dashboard data for the student.
Algorithm	1. Start 2. Define the `StudentAuthController` class, extending the `Controller` base class. 3. Implement the `showLoginForm` method to return the student login view. 4. Implement the `login` method: - Validate credentials (`matric_id` and	

	`password`). - Attempt authentication using `Auth::guard('student')`. - Redirect to the password change page if it is the student's first login. - Otherwise, redirect to the student dashboard. - Return back with errors if authentication fails. 5. Implement the `showChangePasswordForm` method to return the password change view. 6. Implement the `changePassword` method: - Validate the input for current password and new password. - Verify the current password matches the stored password. - Update the password and set `is_first_login` to `false`. - Save the student's updated record. - Redirect to the dashboard with a success message. 7. Implement the `dashboard` method: - Retrieve the authenticated student. - Fetch tasks assigned to the student within the specified timeframe. - Query the database for pending topics and appointments. - Return the dashboard view with tasks, `pendingTopicCount`, and `pendingAppointmentCount`. 8. Implement the `logout` method to log out the

	student and redirect to the login page. 9. End

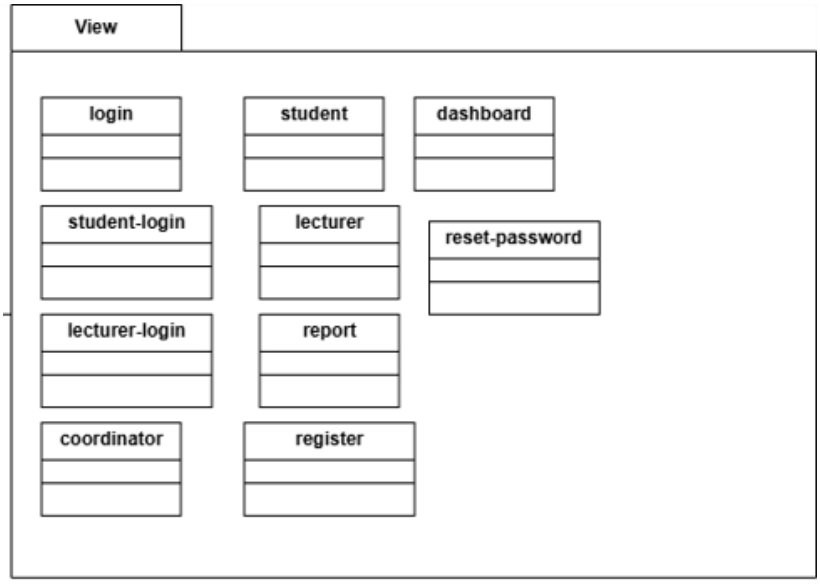


Figure 4.3 View Manage user

4.1.10 [SDD-REQ-110] login.blade.php

Class Type	View	
Responsibility	• Display the login form to the user. • Validate and submit user credentials for authentication.	
Attributes	**Attributes Name**	**Attributes Type**
	email	string
	password	string
	remember_me	boolean

Methods	**Method Name**	**Description**
	login	Authenticate user and redirect.
Algorithm	1. Start 2. Display the login form. 3. Validate user input (email and password). 4. If valid: a. Authenticate user. b. If successful: i. Redirect to the dashboard. ii. If remember_me is true, maintain session. c. Else: i. Display error message. 5. End	

4.1.11 [SDD-REQ-111] student-login.blade.php

Class Type	View	
Responsibility	- Display a login form for students. - Accept student credentials for authentication. - Provide error messages for invalid inputs.	
Attributes	**Attributes Name**	**Attributes Type**
	matric_id	string
	password	string

	csrf_token	string
Methods	**Method Name**	**Description**
	showLoginForm	Render the student login form.
	handleLoginSubmission	Submit the login credentials for authentication.
	displayErrorMessages	Show error messages for invalid credentials.
Algorithm	1. Start 2. Define the view for the student-login.blade.php file. 3. Render the HTML structure with a div centered vertically and horizontally. 4. Include a form element with: - action attribute set to the student login route (route('student.login')). - method attribute set to POST. - @csrf directive for security. 5. Create input fields for: - matric_id with placeholder "Matric ID". - password with placeholder "Password". - Apply Tailwind CSS for styling. 6. Add a button to submit the form with classes for hover and focus effects. 7. Include a section to display validation errors	

	for matric_id using @error directive. 8. End.

4.1.12 [SDD-REQ-112] lecturer-login.blade.php

Class Type	View	
Responsibility	- Display a login form for lecturers. - Accept lecturer credentials for authentication. - Provide error messages for invalid inputs.	
Attributes	**Attributes Name**	**Attributes Type**
	staff_id	string
	password	string
	csrf_token	string
Methods	**Method Name**	**Description**
	showLoginForm	Render the lecturer login form.
	handleLoginSubmission	Submit the login credentials for authentication.

	displayErrorMessages	Show error messages for invalid credentials.
Algorithm	1. Start 2. Define the view for the lecturer-login.blade.php file. 3. Render the HTML structure with a div centered vertically and horizontally. 4. Include a form element with: - action attribute set to the lecturer login route (route('lecturer.login')). - method attribute set to POST. - @csrf directive for security. 5. Create input fields for: - staff_id with placeholder "Staff ID". - password with placeholder "Password". - Apply Tailwind CSS for styling. 6. Add a button to submit the form with classes for hover and focus effects. 7. Include a section to display validation errors for staff_id using @error directive. 8. End.	

4.1.13 [SDD-REQ-113] coordinator.blade.php, student.blade.php & lecturer.blade.php

Class Type	View
Responsibility	To define the structure and layout for the Coordinator, Lecturer and Student dashboard,

	providing navigation and dynamic content rendering for Coordinators, Lecturers and Students.	
Attributes	**Attributes Name**	**Attributes Type**
	title	String
	csrf-token	String
Methods	**Method Name**	**Description**
	renderNavigation	Generates the navigation bar with links specific to Coordinators, Lecturers and Students.
	setTitle	Sets the page title dynamically using @yield('title').
	renderContent	Injects specific content dynamically using @yield('content').
Algorithm	1. Start. 2. Define the Blade file and set up the base layout using @extends. 3. Add placeholders for the title, styles, and scripts using @yield.	

	4. Create a navigation bar specific to Lecturers with links and a logout button. 5. Define a main content section for dynamic content injection using @yield('content'). 6. Implement CSRF protection using @csrf. 7. Apply custom styles and branding elements. 8. End.

4.1.14 [SDD-REQ-114] report.blade.php

Class Type	View	
Responsibility	• Represents an individual lecturer/student and stores their data.	
Attributes	**Attributes Name**	**Attributes Type**
	staff_id/matric id	String
	name	String
	email	String
	research_group/program	String
	Lecturers/students	Array<Lecturer>/ Array<Student>
	generation_date	DateTime

Methods	**Method Name**	**Description**
	getFullName()	Returns the full name of the lecturer.
	getContactInfo()	Returns the lecturer's contact details.
	getResearchGroup()	Retrieves the research group information.
	addLecturer(Lecturer)/ addStudent(Student)	Adds a lecturer/student to the report.
	generateAndExport()	Combines report generation and export functionality.
	getDetails()	Retrieves all details of the student as an array.
	getProgram()	Returns the student's academic program.
Algorithm	8. Start 9. Initialize an empty string html to hold the HTML content. 10. Append the header section of the report, including the title and generation date. 11. Create an HTML table with the following columns: Staff ID, Name, Email, Research Group. (varies for student's report) 12. Iterate over the lecturers array: For each lecturer, create a table row and populate it with the lecturer's details.	

	13. Close the table. 14. If exporting to Word is required: Use a library (e.g., PHPWord) to create a Word document. 15. Populate the Word document with the report data. 16. Save the document to a temporary file and trigger download. 17. End

4.1.15 [SDD-REQ-115] register.blade.php

Class Type	View	
Responsibility	• Display the registration form to the user. • Validate and submit user data for account creation.	
Attributes	**Attributes Name**	**Attributes Type**
	name	string
	email	string
	password	string
	password_confirmation	string
Methods	**Method Name**	**Description**
	register	Create user account and redirect.

Algorithm	1. Start 2. Display the registration form. 3. Validate user input (name, email, password, and password_confirmation). 4. If valid: a. Create user account. b. Redirect to login page or dashboard. 5. End

4.1.16 [SDD-REQ-116] dashboard.blade.php

Class Type	View	
Responsibility	• Provide an overview of statistics and research groups for coordinators. • Display distribution charts for programs. • List research groups with relevant details.	
Attributes	**Attributes Name**	**Attributes Type**
	totalStudents	integer
	totalLecturers	integer
	researchGroups	array
	studentDistribution	array
Methods	**Method Name**	**Description**

	renderDashboard	Display statistics and charts.
Algorithm	1. Start 2. Fetch data for total students and lecturers. 3. Retrieve research groups and their details. 4. Calculate student distribution by program. 5. Render dashboard view with the collected data. 6. End	

4.1.17 [SDD-REQ-117] reset-password.blade.php

Class Type	View	
Responsibility	To define the structure and layout for the Reset Password page, enabling users to securely reset their password using a token-based authentication system.	
Attributes	**Attributes Name**	**Attributes Type**
	token	String
	email	String
	password	String
	password_confirmation	String
Methods	**Method Name**	**Description**

	validateInput()	Validates the user input for email, password, and password confirmation fields.
	submitForm()	Submits the reset password form to the backend route.
	displayValidationErrors()	Dynamically displays validation errors for incorrect or missing input.
Algorithm	**Start** 1. **Initialize the Reset Password Page**: o Load the reset-password.blade.php page using the ResetPasswordPage class. 2. **Display Validation Errors (if any)**: o Check if any validation errors are passed to the page. o If errors exist: ▪ Use <x-validation-errors> to display the errors. 3. **Prepare the Form**: o Render an HTML form with the following fields:	

	▪ **Hidden Field**: Token ▪ Set the value to {{ $request->route('token') }}. ▪ **Email Field**: ▪ Prepopulate the value with old('email', $request->email). ▪ Set required, type="email", and autocomplete="username". ▪ **Password Field**: ▪ Set type="password", required, and autocomplete="new-password". ▪ **Password Confirmation Field**: ▪ Set type="password", required, and autocomplete="new-password". 4. **User Input Validation**: o When the user submits the form: ▪ Validate the following fields:

- token: Ensure it matches the token sent to the user's email.
- email: Ensure it matches a valid user email.
- password: Ensure it meets the minimum security requirements (e.g., length, complexity).
- password_confirmation: Ensure it matches the password field.

5. **Submit Form Data**:
    - If validation passes:
        - Send the form data to the backend using the POST method and the route {{ route('password.update') }}.
    - If validation fails:
        - Return the user to the page with validation error messages.
6. **Process Backend Logic**:
    - In the backend:

	▪ Verify the token is valid and matches the user. ▪ Update the user's password in the database. ▪ Invalidate the reset token to prevent reuse. o If the reset process succeeds: ▪ Redirect the user to the login page with a success message. o If the reset process fails: ▪ Return the user to the reset page with appropriate error messages. 7. **Display Success Message**: o After a successful reset, display a confirmation message on the login page indicating the password has been updated. **End**

4.2 Package 2 [SDD-REQ-200] (Module: Manage Appointment)

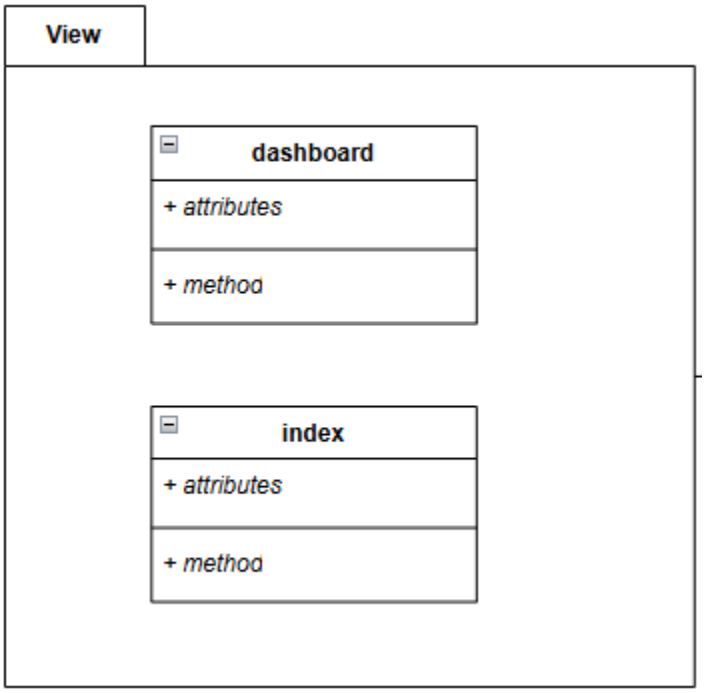


Figure 4.4 View Manage Appointment

4.2.1 [SDD-REQ-201] dashboard.blade.php

Class Type	View	
Responsibility	1. Display an overview of appointment for student and lecturer. - Student Dashboard: pending appointment, timeframe section. - Lecturer Dashboard: pending appointment, timeframe section.	
Attributes	**Attributes Name**	**Attributes Type**
	totalAppointment	integer
	upcomingAppointment	Array/List
	completedAppointments	Integer
	cancelledAppointments	Integer

	appointmentsByCategory	Associative Array
Methods	**Method Name**	**Description**
	displayAppointmentSummary()	Displays a summary of total, upcoming, completed, and canceled appointments in a dashboard widget.
	renderUpcomingAppointments()	Renders a list or table of upcoming appointments on the dashboard, showing essential details like date, time, and participants.
	generateCategoryOverview()	Displays a breakdown of appointments by categories such as lecturers, students a and types of appointments.
	showQuickActions()	Provides quick links or buttons for common actions such as schedule a new

		appointment, view all appointments.
Algorithm	For student and lecturer: 1. Login with Matric ID, lecturer ID 2. Retrieve the total number of appointments from the database. 3. Total appointments, count of upcoming, completed, and cancelled appointments. 4. Group appointments by relevant categories such as lecturers for student view, students for lecturer view. 5. Implement displayAppointmentSummary(): show a summary of total appointments, upcoming, completed, and canceled appointments. 6. Implement renderUpcomingAppointments(): generate a list or table displaying upcoming appointments with details such as date, time, participants. 7. Implement generateCategoryOverview(): display an overview of appointment counts by category. 8. Implement showQuickActions(): provide quick action buttons (e.g., "Schedule Appointment" or "View All Appointments").	

	9. Filter the list of upcoming appointments to display those with the status “Pending”. 10. Highlight appointments happening within a specific timeframe within 1 day. 11. View the summarized data and lists to the dashboard.blade.php view for rendering.

4.2.2 [SDD-REQ-202] index.blade.php

Class Type	View	
Responsibility	List and manage detailed appointment records. It serves as a user-friendly interface for performing CRUD (Create, Read, Update, Delete) operations.	
Attributes	**Attributes Name**	**Attributes Type**
	appointment	Array/List
	actions	Boolen
	filters	Associative Array
	pagination	Object or Array
Methods	**Method Name**	**Description**
	listAppointments()	Displays all appointments in a tabular or list format with key details like date, time, lecturer, and status.

	filterAppointments(filters)	Filters the appointment list based on student and lecturer selected criteria such as date range and lecturer.
	paginateAppointments()	Implements pagination for large lists of appointments to improve navigation and usability.
	renderActionButtons(appointment)	Generates action buttons such as Edit, Delete, View Details for each appointment in the list.
	sortAppointments(criteria)	Sorts appointments based on criteria such as date and lecturer name.
	showAppointmentDetails(appointmentId)	Displays detailed information about a specific appointment when the user clicks "View Details."
	highlightUpcomingAppointments()	Highlights appointments happening soon or within one day before appointment date.

	deleteAppointment(appointmentId)	Provides a method to delete a specific appointment and refresh the list.
Algorithm	For student and lecturer: 1. Query all appointments from the database for the respective student or lecturer. Apply default filters such as show only active appointments. 2. appointment list details such as id, date, time, lecturer, student, and status. 3. Apply filterAppointments(filters): filter appointments based on the criteria provided such as date range and status. 4. Implement paginateAppointments(): that divide the appointment list into manageable pages. 5. Apply listAppointments():to display appointments in a tabular format and include columns for essential details and action buttons. 6. Implement action buttons renderActionButtons(appointment): for each appointment, generate buttons for actions like Edit, Delete, or View Details. 7. Implement sortAppointments(criteria): to enable sorting by date and status. 18. Implement highlightUpcomingAppointments(): that happening soon or within one day of the current date. 19. Implement deleteAppointment(appointmentId) to remove an appointment and refresh the list. 20. Implement showAppointmentDetails(appointmentId) to display detailed information about a specific appointment.	

	21. View appointment list, pagination details, and filters to the index.blade.php view for rendering.

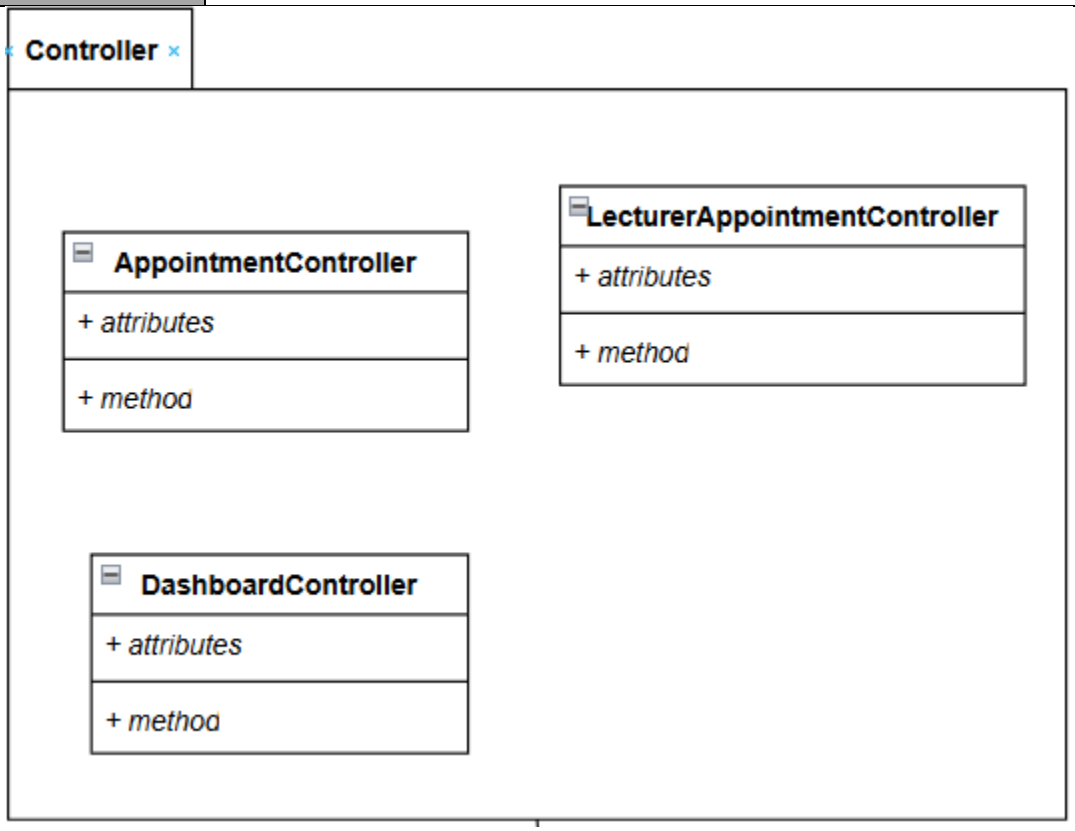


Figure 4.5 Controller Manage Appointment

4.2.3 [SDD-REQ-203] DashboardController.blade.php

Class Type	Controller	
Responsibility	- Handle dashboard related logic and render the dashboard view. - Aggregate data from multiple sources to provide an overview for the user.	
Attributes	**Attributes Name**	**Attributes Type**
	Student	Object or array

	Lecturer	Object or array
	Stats	Collection
Methods	**Method Name**	**Description**
	renderDashboard	Handles rendering the dashboard view with relevant data.
	fetchNotifications	Retrieves recent notifications or alerts for display on the dashboard.
Algorithm	For student and lecturer: 1.Rendering dashboard view 2.Query the database for relevant data: - Dashboard statistics such as total pending appointment. 4.Notifications such as recent messages or alerts. 5. Organize and format the fetched data for rendering. 6. View the data to the dashboard.blade.php view.	

4.2.4 [SDD-REQ-204] AppointmentController.blade.php

Class Type	Controller	
Responsibility	- Manage and render appointment-related views. - Process appointment data between the front-end and back-end.	
Attributes	**Attributes Name**	**Attributes Type**
	appointments	Array and collection.
	Student	Eloquent Model

	Lecturer	Eloquent Model
Methods	**Method Name**	**Description**
	viewAppointments	Fetches and renders the list of appointments.
	createAppointment	Displays a form to create a new appointment.
	storeAppointment	Handles storing a new appointment into the database.
Algorithm	1. List Appointments - Check the user's role such as lecturer, student. - Fetch appointments from the database using appropriate filters. - View the appointments to the appointments.blade.php view. 2. Create New Appointment - Display a form to input new appointment details. - Validate the submitted data: Ensure required fields such as date, time, description are present. Check for conflicts such as overlapping times. 3. Store Appointment - Save the validated data into the database using the Appointment model. - Return a success message or handle errors. 4. Update/Delete an Appointment - Fetch the specific appointment by ID. - Update or delete the record in the database based on user input. 5.	

4.2.5 [SDD-REQ-205] LecturerAppointmentController.blade.php

Class Type	Controller	
Responsibility	Handle appointments specifically for lecturers.	
Attributes	**Attributes Name**	**Attributes Type**
	lecturer	Data object
	appointments	Data object
Methods	**Method Name**	**Description**
	viewLecturerAppointments	Renders appointments specific to a lecturer.
	assignLecturerToAppointment	Assigns a lecturer to a particular appointment.
Algorithm	Manage lecturer specific appointments: 1. login with staff ID as lecturer 2. Apply filters such as upcoming, past, or status 3. Implement data to view for rendering (lecturer_appointments.blade.php). 4. **Assign Lecturer to Appointment by validating** the appointment and lecturer IDs, updating appointment record to link it to the lecturer. 5. Return a confirmation or error message.	

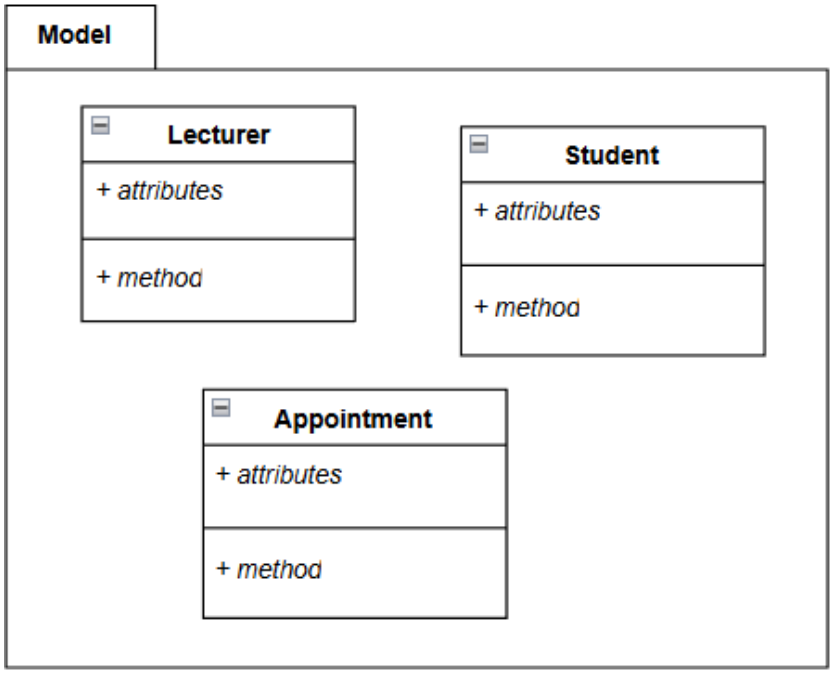


Figure 4.6 Model Manage Appointment

4.2.6 [SDD-REQ-206] Lecturer.php

Class Type	Controller	
Responsibility	- Manage their own availability for appointments. - Allows students to schedule appointments with them. - Maintains their information, such as name, department, and ID.	
Attributes	**Attributes Name**	**Attributes Type**
	lecturerName	String
	lecturerID	Integer
Methods	**Method Name**	**Description**
	scheduleAppointment	Allows the lecturer to set appointments with students.
	getDetails	Returns the lecturer's information.
Algorithm	**Schedule Appointment** 1. **Input**: - Lecturer ID	

	- Student ID - Date - Time 2. **Check Availability**: - Retrieve the lecturer's schedule for the specified date. - If the time slot is occupied: **Output**: "Time slot unavailable. Choose another time." - **End**. 3. **Update Schedule**: - Reserve the specified time slot for the appointment. - Link the student to the appointment. 4. **Save**: - Store the appointment in the database with details. 5. **Output**: "Appointment scheduled successfully."

4.2.7 [SDD-REQ-207] Student.php

Class Type	Model	
Responsibility	- Manage their own appointments with lecturers. - Retrieve appointment history or details.	
Attributes	**Attributes Name**	**Attributes Type**
	studentName	String
	studentID	Integer
Methods	**Method Name**	**Description**
	bookAppointment	Allows the student to book an appointment with a lecturer.
	getDetails	Returns the student's information.
Algorithm	Book Appointment **1. Input**: - Student ID, Date, start Time, end time, purpose of meeting	

	2. **Verify Eligibility**: - Check if the student is eligible to book an appointment (e.g., active enrollment). - If ineligible: **Output**: "Student not eligible to book appointments." - **End**. 3. **Check Lecturer Availability**: - **View Lecturer.ScheduleAppointment** method to verify availability. - If unavailable: **Output**: "Selected time slot is unavailable." - **End**. 4. **Create Appointment**: - Generate an appointment request with the specified details. - Send the request to the **Appointment** class for confirmation. 5. **Output**: "Appointment request sent successfully."

4.2.8 [SDD-REQ-208] Appointment.php

Class Type	Model	
Responsibility	Store and manage details of an appointment, such as date, time, associated lecturer and student ID.	
Attributes	**Attributes Name**	**Attributes Type**
	appointmentID	Integer
	date	Date
	time	Time
	lecturerID	Integer (Foreign Key)
	studentID	Integer (Foreign Key)
Methods	**Method Name**	**Description**

	createAppointment	Registers a new appointment.
	getAppointmentDetails	Retrieves appointment details.
Algorithm	**Create Appointment** 1. **Input**: - Appointment ID (unique, generated automatically), Student ID, Date, start Time, end time, purpose of meeting 2. **Validation**: - Ensure no conflicts exist in the schedule (i.e., the same time slot is not already booked for the lecturer or student). - If a conflict exists: **Output**: "Appointment cannot be created due to scheduling conflict." - **End**. 3. **Record Appointment**: - Save the appointment details in the database: Appointment ID, Lecturer ID, Student ID, Date and Time. 4. **Notify**: - Inform both the lecturer and the student of the appointment details. 5. **Output**: "Appointment successfully created."	

4.3 Package 3 [SDD-REQ-300] (Module 3: Apply Topic)

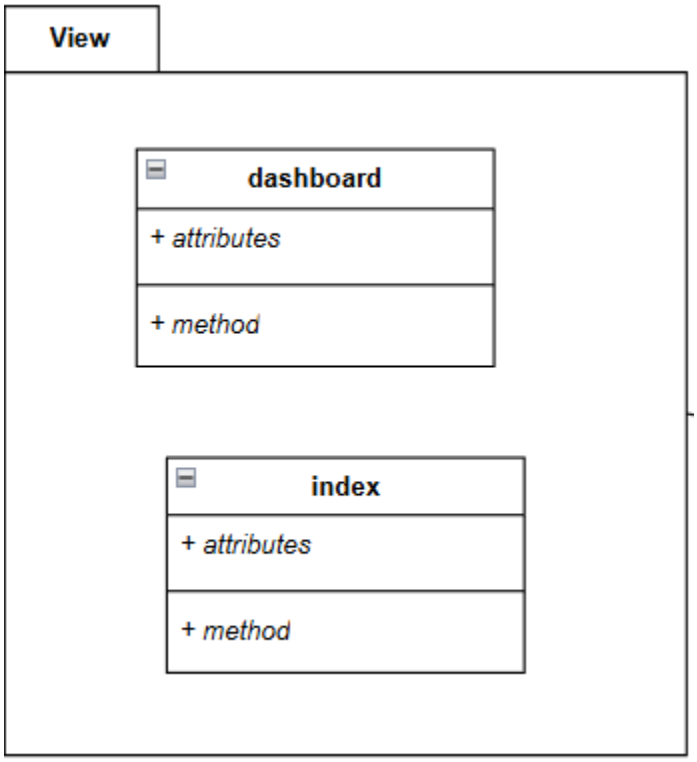


Figure 4.7 View Apply Topic

4.3.1 [SDD-REQ-301] dashboard.blade.php

Class Type	View	
Responsibility	• Displays personalized dashboard information for lecturers, including their details, pending tasks, and current timeline. • Displays the student's dashboard with basic navigation and a welcome message.	
Attributes	**Attributes Name**	**Attributes Type**
	name	String
	staff_id/matric id	String
	research_group/ program	String
	expertise	String

	pendingTopicsCount	Integer
	pendingAppointmentsCount	Integer
	tasks	Array<Object>
Methods	**Method Name**	**Description**
	getLecturerDetails()	Retrieves the lecturer's information (name, staff ID, research group, expertise).
	getPendingTopics()	Retrieves the count of pending topics requiring the lecturer's approval.
	getPendingAppointments()	Retrieves the count of pending appointments for the lecturer.
	getCurrentTimeline()	Retrieves the lecturer's current tasks with their details (name, status, etc.).
	getHeader()	Retrieves the title text for the dashboard header.
	loadWelcomeComponent()	Loads the reusable x-welcome component with the welcome message.
Algorithm	Lecturer Dashboad: 1. Start 2. Initialize the Dashboard 3. Load the dashboard.blade.php page using the LecturerDashboardPage class. 4. Fetch the authenticated lecturer’s details (name, staff ID, research group, expertise). 5. Display Lecturer Info 6. Render a header section showing the lecturer’s name, staff ID, research group, and expertise. 7. Render Summary Cards 8. Create two summary cards: 9. Card 1: Pending Topic Approvals - Show the count retrieved via getPendingTopics(). 10. Card 2: Pending Appointments - Show the count retrieved via getPendingAppointments().	

	11. Display Current Timeline 12. Fetch the tasks using getCurrentTimeline(). 13. For each task: 14. Render the task name, status, start date, and end date. 15. Use the task's color attribute to dynamically style the border and status badge. 16. If no tasks are available, display a placeholder message: "No active tasks at the moment." 17. Finish Rendering 18. Compile all sections and display the final dashboard page to the lecturer. 19. End Student Dashboard: 1. Start 2. Initialize the Dashboard 3. Load the dashboard.blade.php page using the StudentDashboardPage class. 4. Render the Header Section 5. Set the page title to "Dashboard" using the getHeader() method. 6. Include a styled header section with the page title. 7. Display the Main Content 8. Create a container for the main content using a card layout. 9. Load and render the x-welcome component using loadWelcomeComponent(). 10. Finish Rendering 11. Compile all sections and display the final dashboard page to the student. 12. End

4.3.2 [SDD-REQ-302] index.blade.php

Class Type	View
Responsibility	• This file represents the 'Topic Management' page for lecturers. It allows lecturers to view, approve, reject, and provide feedback on student-submitted topics. The page includes

	dynamic content rendering, modals for review and feedback, and session-based notifications. • Display a list of submitted topics for students. • Allow students to manage their topics by adding, editing, or deleting them. • Provide feedback information to students.	
Attributes	**Attributes Name**	**Attributes Type**
	topics	Collection
	success	Session Variable
	topics	Collection
	lecturers	Collection
	success	String
	error	String
	addTopicModal	HTML Element
	feedbackModal	HTML Element
	feedbackContent	String
Methods	**Method Name**	**Description**
	showReviewModal(topicId, title)	Displays the modal for reviewing a topic with the selected ID and title.
	viewFeedback(feedback)	Displays the feedback modal with the given feedback content.
	viewFeedback(feedback)	Displays supervisor feedback in a modal.
	editTopic(id)	Placeholder function to handle the topic editing functionality.

<table>
<tr><td></td><td>document.getElementById</td><td>Used to toggle visibility for modals by manipulating their classList.</td></tr>
<tr><td></td><td>submitTopic()</td><td>Handles the submission of a new topic (via POST request to student.topic.store).</td></tr>
<tr><td>Algorithm</td><td colspan="2">Lecturer Index:
1. Start
2. Check for session variables to display any success messages.
3. Render a table displaying topics:
4. - For each topic, display student name, title, research area, and status.
5. - If the topic is pending, render a "Review" button to open the review modal.
6. - Otherwise, render a "View Feedback" button to show feedback.
7. Handle empty state by showing a placeholder message if no topics are submitted.
8. Implement two modals:
9. - Review Modal: Allows lecturers to update the status and provide feedback.
10. - Feedback Modal: Displays feedback for a previously reviewed topic.
11. Attach JavaScript functions for dynamic modal handling.
12. End

Student Index:
1. Start
2. Render Success/Error Messages:
• If a success message exists in the session, display it in a green-styled block.
• If an error message exists in the session, display it in a red-styled block.
3. Display Topics Table:
• Iterate over the topics collection.
• For each topic:
• Show its title, description (truncated), research area, supervisor, and status (color-coded).
• Include action buttons:
• Edit (if status is pending).
• Delete (if status is pending, with confirmation).
• View Feedback (if feedback exists).</td></tr>
</table>

	• If no topics exist, show a "No topics submitted yet" message. 4. **Handle New Topic Submission**: • Provide an "Add Topic" button to open a modal. • In the modal, collect inputs for: • Title, description, research area, and supervisor. • Submit the form via a POST request to store the new topic. 5. **Handle Feedback Display**: • When "View Feedback" is clicked: • Set the modal content to the feedback text. • Make the modal visible by removing the hidden class. 6. **Modal Visibility Management**: • **Open Modal**: Remove the hidden class from the modal's element. • **Close Modal**: Add the hidden class to hide the modal. 7. **Handle Topic Deletion**: • On "Delete" button click: a. Confirm the deletion action with a popup. b. If confirmed, send a DELETE request to remove the topic. 8. **Placeholder for Editing**: • When "Edit" is clicked, display an alert indicating that the functionality is yet to be implemented. 9. End

4.3.3 [SDD-REQ-302]

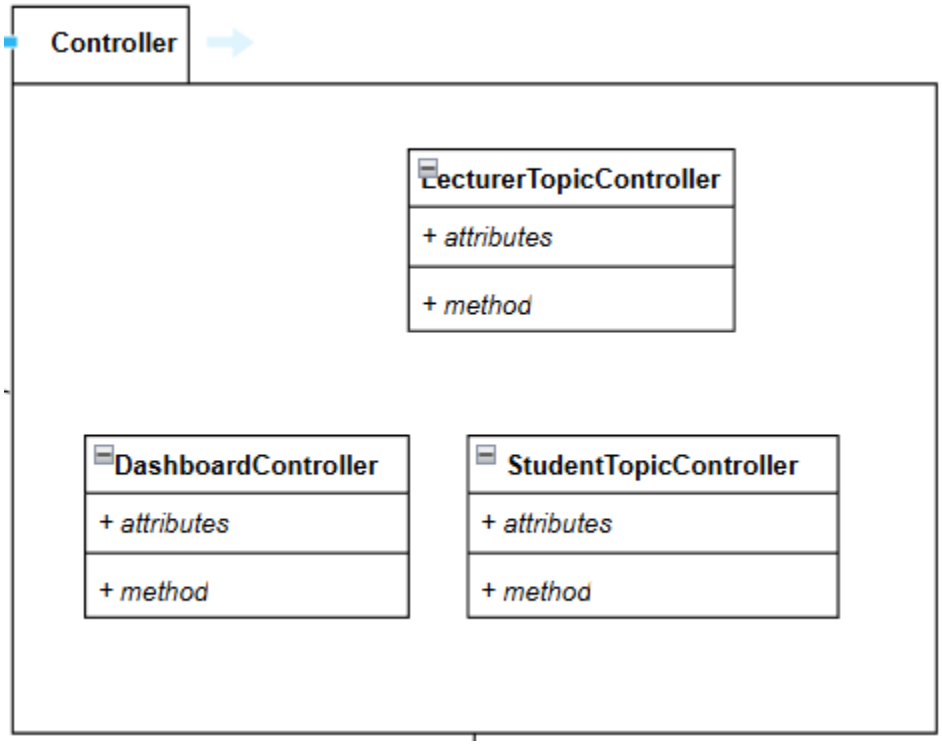


Figure 4.8 Controller Apply Topic

4.3.4 [SDD-REQ-305] LecturerTopicController.php

Class Type	Controller	
Responsibility	• Manage lecturer-related operations on topics such as viewing, updating, and showing topics.	
Attributes	**Attributes Name**	**Attributes Type**
	`$lecturer`	Object
	`$topics`	Object
Methods	**Method Name**	**Description**
	`index()`	Fetches and displays all topics associated with the authenticated lecturer.

	`update()`	Updates the status and feedback of a specific topic based on lecturer input.
	`show()`	Displays detailed information about a specific topic.
Algorithm	1. Authenticate the lecturer using the `Auth` facade. 2. In the `index()` method: a. Retrieve topics associated with the lecturer. b. Include related student data using eager loading. c. Pass the topics to the `index` view. 3. In the `update()` method: a. Validate the request for `status` and `feedback`. b. Update the `Topic` model with the validated data. c. Return a success message. 4. In the `show()` method: a. Load the `student` relationship for the topic. b. Pass the topic data to the `show` view.	

4.3.5 [SDD-REQ-306] StudentTopicController.php

Class Type	Controller	
Responsibility	Manage student-related operations on topics, such as listing, storing, updating, and deleting.	
Attributes	**Attributes Name**	**Attributes Type**
	`$topics`	Object
	`$lecturers`	Object
Methods	**Method Name**	**Description**

	`index()`	Lists topics submitted by the authenticated student and retrieves all lecturers.
	`store()`	Validates and creates a new topic submitted by the student.
	`update()`	Updates a topic if its status is still `pending`.
	`destroy()`	Deletes a topic if its status is still `pending`.
Algorithm	1. Authenticate the student using the `Auth` facade. 2. In the `index()` method: a. Retrieve topics submitted by the authenticated student. b. Fetch all available lecturers. c. Pass the data to the `index` view. 3. In the `store()` method: a. Validate the request for required fields. b. Create a new topic with the provided data. c. Return a success message. 4. In the `update()` method: a. Check if the topic is `pending`. b. Validate and update the topic data. c. Return a success message. 5. In the `destroy()` method: a. Check if the topic is `pending`. b. Delete the topic and return a success message.	

4.3.6 [SDD-REQ-307] DashboardController.php

Class Type	Controller	
Responsibility	Manage lecturer dashboard operations, such as displaying pending topics, appointments, and tasks.	
Attributes	**Attributes Name**	**Attributes Type**
	`$lecturer`	Object
	`$pendingTopicsCount`	Integer
	`$pendingAppointmentsCount`	Integer
	`$tasks`	Object
Methods	**Method Name**	**Description**
	`index()`	Displays the lecturer dashboard with pending topics, appointments, and timeframe tasks.
Algorithm	1. Authenticate the lecturer using the `Auth` facade. 2. In the `index()` method: a. Retrieve the authenticated lecturer. b. Count pending topics assigned to the lecturer. c. Count pending appointments for the lecturer. d. Fetch tasks within the current timeframe. e. Pass all data to the dashboard view.	

4.3.7 [SDD-REQ-305]

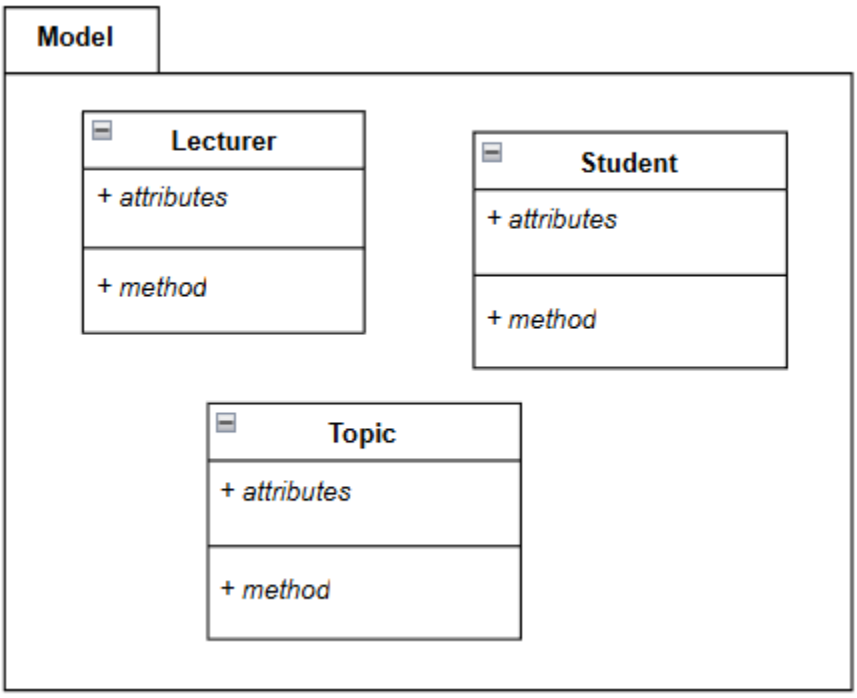


Figure 4.9 Model Apply Topic

4.3.8 [SDD-REQ-306] Lecturer.php

Class Type	Model	
Responsibility	• Represent and manage the data for a lecturer in the system. • Handle lecturer authentication, including password storage and first login status. • Define attributes and their behavior (e.g., casts, hidden). • Manage the relationship with the Quota model.	
Attributes	**Attributes Name**	**Attributes Type**
	staff_id	string
	name	string

	email	string
	research_group	string
	password	string
	is_first_login	boolean
Methods	**Method Name**	**Description**
	quota()	Defines a one-to-one relationship between Lecturer and Quota. This allows access to the lecturer's assigned quota data.
Algorithm	1. Start 2. Define the Lecturer class, extending the Authenticatable class. 3. Specify the fillable attributes: staff_id name email research_group password is_first_login 4. Mark the password attribute as hidden to protect it in JSON responses. 5. Cast the is_first_login attribute to a boolean for easier management. 6. Define a one-to-one relationship with the Quota model using the quota() method. 7. End	

4.3.9 [SDD-REQ-307] Student.php

Class Type	Model	
Responsibility	• Represent and manage the data for a student in the system. • Handle student authentication, including password storage and first login status. • Define attributes and their behavior (e.g., casts, hidden).	
Attributes	**Attributes Name**	**Attributes Type**
	matric_id	String
	name	String
	email	String
	program	String
	password	String (Hidden)
	is_first_login	Boolean
Methods	**Method Name**	**Description**
	quota()	Defines a one-to-one relationship between Student and Quota. This allows access to the

		student's assigned quota data.
Algorithm	8. **Start** 9. Define the Student class, extending Authenticatable to manage student authentication. 10. Specify the fillable attributes: 1. matric_id 2. name 3. email 4. program 5. password 6. is_first_login 11. Mark the password attribute as hidden to ensure it's not included in array or JSON responses. 12. Cast the is_first_login attribute to a boolean for proper data handling. 13. Define the quota() method to establish a one-to-one relationship with the Quota model. 14. **End**	

4.3.10 [SDD-REQ-308] Topic.php

Class Type	Model	
Responsibility	• Represent the topics table in the database. • Define relationships between topics, students, and lecturers. • Manage topic attributes such as title, description, status, and more.	
Attributes	**Attributes Name**	**Attributes Type**
	student_id	Integer
	lecturer_id	Integer
	title	String
	description	String
	status	String
	feedback	String
	research_area	String
	timestamps	Timestamps
Methods	**Method Name**	**Description**
	student()	Defines a one-to-one relationship between the `Topic` and the `Student` model.
	lecturer()	Defines a one-to-one relationship between the `Topic` and the `Lecturer` model.
Algorithm	1. **Define Relationships:**	

	- Use the belongsTo method to establish relationships with the Student and Lecturer models. 2. **Handle Attributes:** - Utilize the fillable property to control which attributes can be mass assigned. - Allow Eloquent to automatically manage created_at and updated_at timestamps. 3. **Database Operations:** • To fetch topics: $topics = Topic::where('lecturer_id', $lecturerId)->get(); • To update a topic: $topic->update(['status' => 'approved']);

4.4 Package 4 [SDD-REQ-400] (Module: Manage Timeframe and Quota)

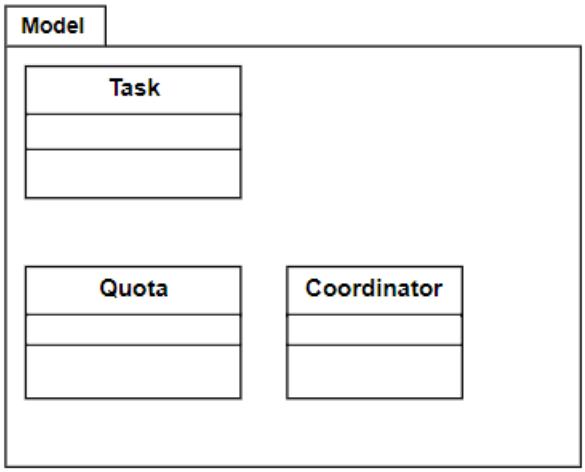


Figure 4.10 Model Manage Timeframe and Quota

4.4.1 [SDD-REQ-401] Coordinator.blade.php

Class Type	Model	
Responsibility	Represents the Coordinator entity in the database. Manages authentication and notification functionalities for Coordinators.	
Attributes	**Attributes Name**	**Attributes Type**
	$fillable	array
	$hidden	array
	email	string
	password	string
	remember_token	nullable string
Methods	**Method Name**	**Description**

	save()	Saves the current Coordinator instance to the database.
	update()	Updates existing Coordinator data in the database.
	delete()	Deletes the Coordinator record from the database.
	notify($notification)	Sends a notification to the Coordinator using the Notifiable trait.
Algorithm	1. Start. 2. Define Coordinator attributes: email (String) and password (String). 3. Validate input data: ensure email is unique and password is secure. 4. Save the Coordinator using save(). 5. Update Coordinator data (if needed) using update(). 6. Delete Coordinator data (if needed) using delete(). 7. Notify Coordinator about system updates using notify($notification).	

	8. End.

4.4.2 [SDD-REQ-402] Task.blade.php

Class Type	Model	
Responsibility	Represents a task or activity linked to a time frame, including time frame name and description.	
Attributes	**Attributes Name**	**Attributes Type**
	id	bigint
	start_date	datetime
	end_date	datetime
	description	text
Methods	**Method Name**	**Description**
	validate()	Ensures that the start date is before the end date and within valid ranges.
	save()	Saves the time frame data to the database.
	update()	Updates existing task details.
	delete()	Deletes the time frame entry from the database.

Algorithm	1. Start. 2. Define Task attributes: start_date (Datetime), end_date (Datetime), and description (Text). 3. Validate that start_date is earlier than end_date. 4. Save Task data using save(). 5. Update Task data as needed using update(). 6. Delete Task from the database using delete(). 7. End.

4.4.3 [SDD-REQ-403] quota.blade.php

Class Type	Model	
Responsibility	Represents students and their associated data, such as matric ID, assigned lecturer for quota.	
Attributes	**Attributes Name**	**Attributes Type**
	id	bigint
	lecturer_id	bigint
	max_supervisees	integer
	current_supervisees	integer

Methods	**Method Name**	**Description**
	assignStudent()	Assigns a student to a lecturer's quota, updating the current_supervisees.
	updateQuota()	Updates the maximum supervision limit for a lecturer.
	delete()	Deletes quota data from the database.
Algorithm	1. Start. 2. Define Quota attributes: lecturer_id (Bigint), max_supervisees (Integer), current_supervisees (Integer). 3. Assign a student using assignStudent(). 4. Validate that current_supervisees does not exceed max_supervisees. 5. Update quota limits using updateQuota(). 6. Delete quota data as needed using delete(). 7. End.	

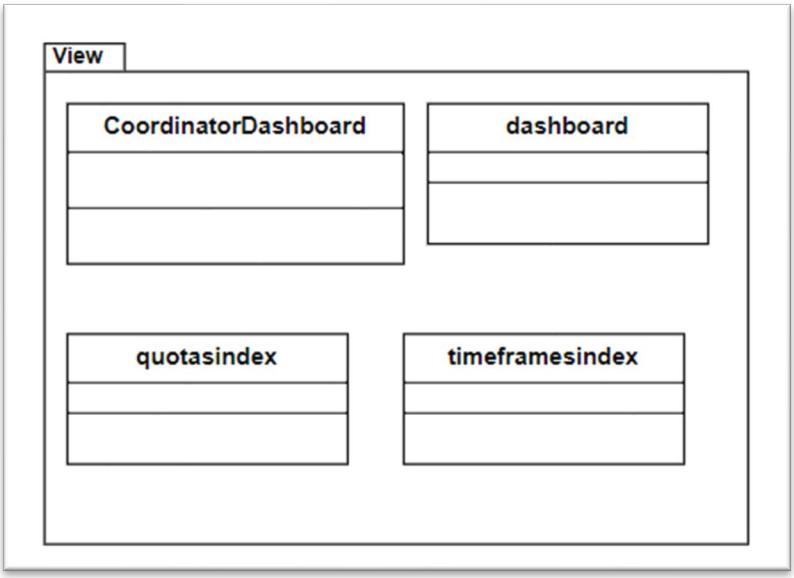


Figure 4.11 View Manage Timeframe and Quota

4.4.4 [SDD-REQ-404] CoordinatorDashboard.blade.php

Class Type	View	
Responsibility	Displays the coordinator's dashboard interface, including an overview of time frames, quotas, and lecturer assignments.	
Attributes	**Attributes Name**	**Attributes Type**
	$index	object
Methods	**Method Name**	**Description**
	renderDashboard()	Fetches the overview data (time frames, quotas, and assignments) from the controller and displays it in the dashboard view.

Algorithm	1. Start. 2. Fetch data from CoordinatorController for: - Time frames overview. - Quotas overview. - Lecturer assignments. 3. Pass the data to the view template. 4. Render the dashboard interface using the fetched data. 5. End.

4.4.5 [SDD-REQ-405] TimeframesIndex.blade.php

Class Type	View	
Responsibility	Manages and displays the list of all defined time frames. Provides options to add, update, or delete entries interactively.	
Attributes	**Attributes Name**	**Attributes Type**
	TaskController	Object
Methods	**Method Name**	**Description**

	renderTimeframes()	Fetches the list of all time frames and displays it in a tabular format.
	addTimeFrame()	Renders a form to add a new time frame. Submits data to the TimeFrameController.
	updateTimeFrame()	Renders a form to update an existing time frame. Submits changes to the TimeFrameController.
	deleteTimeFrame()	Triggers the deletion of a time frame by calling the TimeFrameController.
Algorithm	1. Start. 2. Fetch the list of time frames from the TimeFrameController. 3. Render the time frames in a table format. 4. Add a time frame: - Render a form for the coordinator to input start date, end date, and description. - Submit the data to the addTimeFrame() method of the TimeFrameController.	

	5. Update a time frame: - Render a form pre-filled with the existing time frame data. - Submit the updated data to the updateTimeFrame() method of the TimeFrameController. 6. Delete a time frame: - Call the deleteTimeFrame() method of the TimeFrameController with the selected time frame ID. 7. End.

4.4.6 [SDD-REQ-406] QuotasIndex.blade.php

Class Type	View	
Responsibility	Presents a view for managing lecturer quotas, allowing the coordinator to set or adjust quotas interactively.	
Attributes	**Attributes Name**	**Attributes Type**
	$Quota	Object
Methods	**Method Name**	**Description**
	renderQuotas()	Displays the list of lecturers with their current and maximum quotas.
	adjustQuota()	Renders a form to adjust a lecturer's quota. Updates

		the data using the QuotaController.
Algorithm	1. Start. 2. Fetch the list of lecturers and their quotas (current and maximum) from the QuotaController. 3. Render the quotas in a tabular format. 4. Adjust a quota: - Render a form for the coordinator to input a new maximum quota for a selected lecturer. - Submit the data to the adjustQuota() method of the QuotaController. 5. End.	

4.4.7 [SDD-REQ-407] Dashboard.blade.php

Class Type	View	
Responsibility	A general-purpose dashboard for lecturers and students. Displays key system data, including notifications of updates to quotas and time frames.	
Attributes	**Attributes Name**	**Attributes Type**
	$task	Object
Methods	**Method Name**	**Description**

	renderDashboard()	Fetches and displays notifications and updates related to quotas and time frames.
Algorithm	1. Start. 2. Fetch the latest notifications (e.g., time frame updates, quota changes) from the respective controllers (QuotaController and TimeFrameController). 3. Pass the notifications and updates to the view template. 4. Render the dashboard interface with the fetched data. 5. End.	

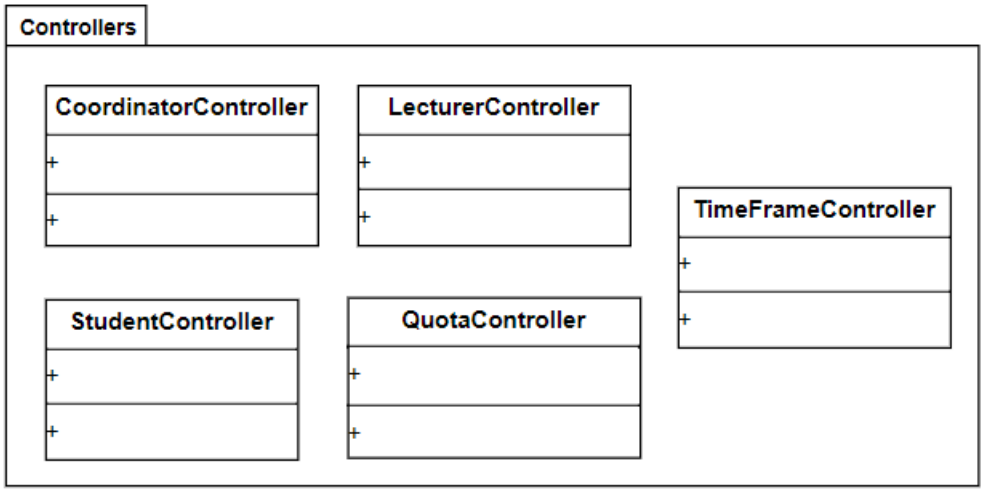


Figure 4.12 Controller Manage Timeframe and Quota

4.4.8 [SDD-REQ-408] CoordinatorController.blade.php

Class Type	Controllers

Responsibility	Oversees coordinator-specific actions, such as modifying time frames and quotas, and managing system-wide settings.	
Attributes	**Attributes Name**	**Attributes Type**
	id	bigint
	email	string
	password	string
	remember_token	nullable string
Methods	**Method Name**	**Description**
	modifyTimeFrame()	Invokes TimeFrameController to manage time frame operations.
	modifyQuota()	Invokes QuotaController to manage quota operations.
Algorithm	1. Start. 2. Fetch the coordinator's input for modifying time frames, quotas, or system settings. 3. Delegate actions to the corresponding controllers (TimeFrameController, QuotaController).	

	4. Perform coordinator-specific settings updates if required. 5. Return success or error response. 6. End.

4.4.9 [SDD-REQ-409] TimeFrameController.blade.php

Class Type	Controllers	
Responsibility	Handles operations related to time frames, such as adding, updating, deleting, and validating time frame data.	
Attributes	**Attributes Name**	**Attributes Type**
	id	bigint
	start_date	date
	end_date	date
	description	string
Methods	**Method Name**	**Description**
	createTimeFrame()	Validates input and creates a new time frame in the database.

	updateTimeFrame()	Validates input and updates an existing time frame.
	deleteTimeFrame()	Deletes a specific time frame using its unique identifier.
	validateTimeFrame()	Checks the validity of the provided time frame data (e.g., date ranges).
Algorithm	1. Start. 2. Validate the input data for time frames (start date, end date, description) using validateTimeFrame(). 3. Perform the corresponding action: - Create: Save the time frame using the createTimeFrame() method of the TimeFrame model. - Update: Locate the time frame by ID and update its attributes. - Delete: Locate the time frame by ID and remove it from the database. 4. Return success or error response. 5. End.	

4.4.10 [SDD-REQ-410] QuotaController.blade.php

Class Type	Controllers	
Responsibility	Manages operations related to quotas, including setting supervision limits, updating lecturer quotas, and validating quota data.	
Attributes	**Attributes Name**	**Attributes Type**
	id	bigint
	lecturer_id	bigint
	max_supervisees	integer
	current_supervisees	integer
Methods	**Method Name**	**Description**
	createQuota()	Validates and creates a new quota for a lecturer.
	updateQuota()	Updates an existing quota record.
	deleteQuota()	Deletes a specific quota using its unique identifier.
	validateQuota()	Ensures that the input quota data (e.g., max supervisees) is valid.

	createQuota()	Validates and creates a new quota for a lecturer.
Algorithm	1. Start. 2. Validate the quota input data (e.g., maximum supervisees, assigned lecturer ID) using validateQuota(). 3. Perform the corresponding action: - Create: Save the new quota using the createQuota() method of the Quota model. - Update: Locate the quota by ID and modify its attributes. - Delete: Locate the quota by ID and remove it from the database. 4. Return success or error response. 5. End.	

4.4.11 [SDD-REQ-411] LecturerController.blade.php

Class Type	Controllers	
Responsibility	Manages lecturer-specific functionalities, such as retrieving assigned quotas and accessing time frame data.	
Attributes	**Attributes Name**	**Attributes Type**
	Lecturer	string

	id	bigint
	name	string
	email	string
	specialization	string
Methods	**Method Name**	**Description**
	getAssignedQuotas()	Fetches the quotas assigned to a specific lecturer.
	getTimeFrames()	Retrieves the time frame data associated with the lecturer.
Algorithm	1. Start. 2. Fetch the lecturer's ID and request assigned data (quotas or time frames). 3. Interact with the respective models (Quota or TimeFrame) to retrieve data. 4. Return the fetched data to the view. 5. End.	

4.4.12 [SDD-REQ-412] StudentController.blade.php

Class Type	Controllers

Responsibility	Handles student-related interactions, including viewing assigned time frames and receiving notifications about updates.	
Attributes	**Attributes Name**	**Attributes Type**
	id	bigint
	name	string
	email	string
	assigned_time_frame	bigint
Methods	**Method Name**	**Description**
	viewTimeFrames()	Fetches and displays the time frames assigned to a student.
	receiveNotifications()	Retrieves and displays notifications related to time frames and quotas.
Algorithm	1. Start. 2. Fetch the student's ID and request the assigned data (time frames or notifications). 3. Interact with the respective models (TimeFrame or Notification) to retrieve data. 4. Display the fetched data to the student in the view.	

	5. End.

5. REQUIREMENT TRACEBILITY

Requirement ID	Description	Design ID
SRS_REQ_101	**Manage User:** • **Register users** – Allows coordinator to do bulk registration for students and lecturers. • **Generate User Reports** - Provides coordinator with detailed student and lecturer reports.	SDD-REQ-100
SRS_REQ_102	**Manage Appointment:** • **Upload Appointment Slots** - Enables lecturers to upload their available appointment timings.	SDD-REQ-200

	• **Request Appointment Pending -** Allows students to request appointments awaiting confirmation. • **View Appointment Slots** - Lets users view all available appointment slots. • **Apply Appointment** - Facilitates students in booking appointment slots with lecturers.	
SRS_REQ_103	**Apply Topic/Title:** • **Manage FYP Topic** - Enables lecturers to manage and edit available FYP topics. • **Apply FYP Topic** - Allows students to apply for their desired FYP topics. • **Respond Application** - Enables lecturers to approve or reject FYP topic applications. • **View Application Status** - Allows students to track the status of their FYP topic application. • **Search and Sort Lecturer** - Helps students find and sort lecturers based on various criteria.	SDD-REQ-300
SRS_REQ_104	**Manage Time Frame and Quota** • **Add Time Frame** - Allows administrators to define the timeline for specific events or processes.	SDD-REQ-400

	• **Update Time Frame** - Enables modifications to previously set time frames. • **Delete Time Frame** - Facilitates the removal of outdated or incorrect time frames. • **Set Lecturer Quota** - Allows administrators to define the maximum number of students per lecturer. • **Dashboard View** – Provides a comprehensive view of system statuses. • **Send Notification** - Sends notifications to users about important updates or changes.	